Universidade do Minho

Licenciatura em Ciências da Computação

Programação Orientada a Objetos

2024/2025

SpotifUM

17 de maio de 2025

João Silva A108483
Matilde Domingues A98982
Miguel Silva A109069

Conteúdo

1 Introdução **3**

2 Classes **4**

2.1 Música . . . 4
2.2 Música Explícita . . . 4
2.3 Música Multimédia . . . 5
2.4 Reprodução . . . 6
2.5 Álbum . . . 6
2.6 Playlist . . . 7
2.7 PlaylistAleatoria . . . 7
2.8 PlaylistFavoritos . . . 7
2.9 PlaylistPersonalizada . . . 8
2.10 PlaylistPorGeneroEDuracao . . . 8
2.11 Utilizador . . . 8
2.12 UtilizadorOcasional . . . 9
2.13 UtilizadorPremium . . . 9
2.14 Subscrição Free . . . 10
2.15 Subscrição Premium Base . . . 10
2.16 Subscrição Premium Top . . . 11
2.17 Subscrição . . . 11
2.18 GestorAlbuns . . . 12
2.19 GestorEstatisticas . . . 13
2.20 GestorMusicas . . . 14
2.21 GestorPersistencia . . . 15
2.22 GestorPlaylists . . . 16
2.23 GestorRecomendacoes . . . 18
2.24 GestorReproducao . . . 19
2.25 GestorUtilizadores . . . 20
2.26 AlbumUI . . . 21
2.27 ReproducaoUI . . . 22
2.28 BibliotecaUI . . . 23
2.29 EstadoUI . . . 24
2.30 EstatisticasUI . . . 25
2.31 LoginUI . . . 27
2.32 Menu . . . 28
2.33 MenuBase . . . 29
2.34 MenuPrincipal . . . 30
2.35 MusicaUI . . . 31
2.36 PlayerUI . . . 32
2.37 PlaylistUI . . . 33
2.38 RecomendacoesUI . . . 33
2.39 UtilizadorUI . . . 34
2.40 ConsoleUtil . . . 35
2.41 DadosImportacao . . . 35
2.42 InputUtil . . . 36

2.43 MusicaDTO 37
2.44 Persistencia 37
2.45 PlaylistDTO 38
2.46 UtilizadorDTO 38
2.47 ReproducaoDTO 39

3 Diagrama de Classes **40**

4 Conclusão **42**

1 Introdução

Este relatório tem como objetivo apresentar uma análise do projeto criado em Java, utilizando programação orientada a objetos. O projeto consiste no desenvolvimento de uma aplicação (SpotifUM) que permite a gestão de músicas, utilizadores e playlists, simulando uma plataforma de reprodução de músicas.

A POO é uma abordagem no desenvolvimento de software que se foca no encapsulamento de dados e dos seus comportamentos relacionados num objeto. Esta metodologia baseia-se em princípios fundamentais como o encapsulamento, herança, polimorfismo e abstração, os quais foram explorados e aplicados ao longo do desenvolvimento da aplicação. Deste modo, vamos explorar a programação orientada a objetos neste projeto, analisando os conceitos fundamentais da POO e a sua aplicação prática.

Assim sendo, iniciaremos este relatório com esta breve introdução, seguida de uma análise às classes criadas, assim como à sua arquitetura. No terceiro capítulo, apresentamos os diagramas de classes. Posteriormente, no quarto capítulo, será abordada a estrutura adotada para desenvolver a aplicação, assim como as funcionalidades implementadas. Por fim, expomos uma conclusão sobre o processo de desenvolvimento do projeto, as dificuldades encontradas e as soluções aplicadas.

2 Classes

2.1 Música

Variáveis de Instância:

- **private String nome;** Armazena o nome da música. Esta variável identifica o título da faixa musical.
- **private String interprete;** Guarda o nome do intérprete da música, ou seja, o artista ou banda que a executa.
- **private String editora;** Indica a editora responsável pela produção ou distribuição da música.
- **private String letra;** Contém a letra da música, que pode ser exibida durante a reprodução.
- **private String genero;** Especifica o género musical da faixa, como pop, rock, jazz, entre outros.
- **private int duracao;** Representa a duração da música em segundos. Esta informação é útil para controlo de tempo de reprodução.
- **private int reproducoes;** Guarda o número de vezes que a música foi reproduzida. Este valor inicia a 0 e é incrementado sempre que a música é tocada, através do método 'incrementarReproducoes()'.
- **private static final int PONTOS_PADRAO = 1;** Constante que representa o número de pontos atribuídos por cada reprodução da música. Este valor é fixo e comum a todas as instâncias da classe.

A classe 'Musica' tem como objetivo central representar e gerir os dados de uma música no sistema. As suas variáveis de instância armazenam as principais características de uma música, e os métodos permitem aceder, modificar e interagir com estas características. Esta classe é fundamental para funcionalidades como reprodução de músicas, exibição de informações musicais e cálculo de estatísticas relacionadas ao número de reproduções.

2.2 Música Explícita

Variáveis de Instância :

- **private String nome;** Guarda o título da faixa.
- **private String interprete;** Artista ou banda que executa a música.
- **private String editora;** Editora responsável pela produção/distribuição.
- **private String letra;** Letra da música, para exibição opcional.
- **private String genero;** Género musical (pop, rock, jazz, ...).

- **private int duracao;** Duração em segundos.
- **private int reproducoes;** Número de reproduções efectuadas.
- **private static final int PONTOS_PADRAO = 1;** Pontos por reprodução.

Descrição da Classe:
A classe `MusicaExplicita` estende `Musica` para representar faixas com conteúdo explícito. Embora não introduza novas variáveis de instância, altera o comportamento dos métodos principais:

- `reproduzir()`: exibe o aviso " AVISO: Conteúdo explícito" antes de delegar para o método da classe-mãe.
- `toString()`: acrescenta o sufixo `[EXPLÍCITA]` à representação textual da música, distinguindo-a em listagens.

2.3 Música Multimédia

Variáveis de Instância :

- **private String nome;** Título da faixa.
- **private String interprete;** Artista ou banda que a interpreta.
- **private String editora;** Editora responsável pela produção/distribuição.
- **private String letra;** Letra da música (para exibição opcional).
- **private String genero;** Género musical (pop, rock, jazz, ...).
- **private int duracao;** Duração em segundos.
- **private int reproducoes;** Número de vezes que a faixa foi reproduzida.
- **private static final int PONTOS_PADRAO = 1;** Pontos atribuídos por reprodução.
- **private String linkVideo;** URL para um vídeo oficial ou lyric video associado à música.

Descrição da Classe:
A classe `MusicaMultimedia` estende `Musica` para incorporar conteúdo audiovisual. Mantém toda a lógica base da classe-mãe e acrescenta:

- `linkVideo` – campo que associa a música a um recurso vídeo (YouTube, Vimeo, *etc.*).
- `reproduzir()`: após executar a reprodução normal (`super.reproduzir()`), informa o utilizador da existência de vídeo: `Video disponível em: <link>`.
- `getLinkVideo()`: método de acesso (getter) que devolve o URL guardado.

Esta especialização é útil para aplicações que mostrem clips, *lyrics videos* ou performances ao vivo juntamente com a faixa de áudio.

2.4 Reprodução

Variáveis de Instância:

- **private final Musica musica;** Referência imutável à faixa reproduzida, permitindo o acesso a todos os seus metadados.
- **private final LocalDateTime dataHora;** Regista a data e hora exactas em que a reprodução ocorreu (fuso horário do sistema).
- **private final String emailUtilizador;** E-mail do utilizador que efectuou a reprodução, útil para estatísticas e histórico pessoal.

Descrição da Classe:
`Reproducao` representa um evento de escuta individual no sistema. A classe é marcada como `Serializable` para que instâncias possam ser persistidas (por exemplo, em ficheiros ou bases de dados). Ao ser criada, guarda:

1. a música (`musica`) passada no construtor;
2. o momento actual (`dataHora = LocalDateTime.now()`);
3. o identificador do utilizador que desencadeou a acção (`emailUtilizador`).

Os métodos `get()` facultam acesso de leitura a estes campos, mantendo-os imutáveis. O `toString()` devolve uma linha legível que combina data/hora, título da música e utilizador — formato conveniente para logs ou relatórios.

2.5 Álbum

Variáveis de Instância:

- **private String nome;** Armazena o nome do álbum. Identifica o título do conjunto de músicas.
- **private String artista;** Guarda o nome do artista responsável pelo álbum.
- **private String editora;** Indica a editora responsável pela publicação do álbum.
- **private List<Musica> musicas;** Lista de músicas que pertencem ao álbum. Cada música é um objeto da classe 'Musica'.

A classe Album tem como objetivo representar uma coleção de músicas associadas a um mesmo artista e editora. Permite adicionar músicas ao álbum, aceder à lista de músicas (com proteção contra modificações externas) e calcular a duração total do álbum. É essencial para organizar e agrupar músicas de forma lógica e estruturada na aplicação.

2.6 Playlist

Variáveis de Instância:

- **private String nome;** Armazena o nome da playlist. Identifica-a de forma única no sistema.
- **protected List<Musica> musicas;** Lista de músicas que compõem a playlist. Permite armazenar múltiplas instâncias da classe 'Musica'.
- **private int reproducoes;** Guarda o número de vezes que a playlist foi reproduzida.
- **protected boolean publica;** Indica se a playlist é pública (visível para outros utilizadores) ou privada.
- **private String criador;** Identifica o utilizador ou sistema que criou a playlist.

A classe Playlist representa uma coleção personalizada de músicas organizada por um utilizador ou pelo sistema. Permite adicionar e remover músicas, consultar informações como número de músicas e total de reproduções, e reproduzir todas as músicas da lista. A possibilidade de tornar a playlist pública ou privada permite controlar a sua visibilidade na aplicação. É essencial para a funcionalidade de personalização e partilha musical entre utilizadores.

2.7 PlaylistAleatoria

Variáveis de Instância:

- **(Herda da classe `Playlist`)** A classe herda todas as variáveis de instância da classe Playlist: nome, musicas, reproducoes, publica e criador.

A classe PlaylistAleatoria estende a classe Playlist e representa uma playlist cuja reprodução é feita de forma aleatória. Ao sobrepor o método reproduzir(), garante que as músicas são tocadas numa ordem aleatória a cada execução, proporcionando uma experiência de audição diferente sempre que a playlist é reproduzida. Esta funcionalidade é útil para utilizadores que preferem variedade na sequência de reprodução das músicas.

2.8 PlaylistFavoritos

Variáveis de Instância:

- **(Herda da classe `Playlist`)** A classe herda todas as variáveis de instância da classe Playlist: nome, musicas, reproducoes, publica e criador.

A classe PlaylistFavoritos é uma extensão da classe Playlist e representa uma playlist personalizada com músicas favoritas. No momento da sua criação, recebe uma lista de músicas recomendadas que são automaticamente adicionadas. Define-se como uma playlist privada (publica = false), não visível a outros utilizadores. Ao reproduzir, exibe uma mensagem específica e toca todas as músicas na ordem em que foram adicionadas. É útil para guardar e aceder rapidamente às músicas preferidas de um utilizador.

2.9 PlaylistPersonalizada

Variáveis de Instância:

- **(Herda da classe `Playlist`)** A classe herda todas as variáveis de instância da classe Playlist: nome, musicas, reproducoes, publica e criador.
- **private int posicaoAtual;** Controla a posição atual da música dentro da playlist, permitindo navegação para a frente e para trás.
- **private boolean aleatoria;** Indica se a reprodução da playlist será feita de forma aleatória ou sequencial.

A classe PlaylistPersonalizada estende a classe Playlist e permite ao utilizador criar uma playlist com controlo adicional sobre a reprodução. Pode ser pública ou privada e suporta reprodução aleatória conforme definido no momento da criação. Os métodos avancar() e retroceder() permitem navegar pelas músicas da lista de forma manual. Esta classe oferece flexibilidade extra, combinando personalização, controlo e aleatoriedade na gestão das músicas.

2.10 PlaylistPorGeneroEDuracao

Variáveis de Instância:

- **(Herdadas da classe `Playlist`)** A classe herda todas as variáveis de instância da classe 'Playlist': 'nome', 'musicas', 'reproducoes', 'publica' e 'criador'.

A classe 'PlaylistPorGeneroEDuracao' estende a classe 'Playlist' e permite criar uma playlist automaticamente com base num género musical e numa duração máxima definida em segundos. No momento da criação, percorre uma lista de músicas disponíveis e adiciona apenas aquelas que pertencem ao género especificado, sem ultrapassar o tempo total definido. Esta playlist é sempre privada. É útil para gerar seleções musicais temáticas e com duração limitada, como para treinos, eventos ou momentos específicos do dia.

2.11 Utilizador

Variáveis de Instância:

- **private String nome;** Armazena o nome do utilizador. Identifica o utilizador na aplicação.
- **private String email;** Contém o endereço de e-mail associado à conta do utilizador.
- **private String morada;** Guarda a morada do utilizador, podendo ser usada para fins administrativos ou de perfil.
- **private String senha;** Guarda a senha de acesso do utilizador. É usada para autenticação no sistema.

- **private BigDecimal pontos;** Representa o total de pontos acumulados pelo utilizador, que podem depender do seu plano de subscrição e número de reproduções.
- **private List<Musica> historicoReproducoes;** Lista que armazena o histórico de músicas reproduzidas pelo utilizador.
- **private LocalDateTime dataRegistro;** Regista a data e hora em que o utilizador foi criado no sistema.

A classe Utilizador é uma classe abstrata que define os atributos e comportamentos comuns a todos os tipos de utilizadores da aplicação. É responsável por armazenar os dados do perfil, controlar o histórico de reproduções e calcular os pontos obtidos com base no comportamento de escuta. Através do método abstrato adicionarPontos(), permite que subclasses definam regras específicas de acumulação de pontos. Esta classe serve como base para diferentes tipos de utilizadores, permitindo uma gestão flexível e segura da informação pessoal e interações com o sistema.

2.12 UtilizadorOcasional

Variáveis de Instância:

- Esta classe não introduz novas variáveis de instância. Herda todas as propriedades da classe abstrata `Utilizador`.

A classe `UtilizadorOcasional` representa um tipo específico de utilizador que utiliza a aplicação de forma pontual. Esta subclasse especializa o comportamento do método `adicionarPontos()`, atribuindo 5 pontos por cada música reproduzida. Ao herdar de `Utilizador`, mantém todos os atributos essenciais (nome, email, senha, pontos, histórico, etc.) e adapta a forma como os pontos são acumulados, distinguindo-se assim dos outros tipos de utilizadores com subscrição.

2.13 UtilizadorPremium

Variáveis de Instância:

- **private Subscricao plano;** Representa o plano de subscrição associado ao utilizador premium. Este plano determina, entre outros aspetos, os pontos iniciais atribuídos e a lógica de cálculo de pontos com base na utilização da aplicação.

A classe `UtilizadorPremium` estende a classe `Utilizador` e representa um utilizador com acesso a funcionalidades premium. Ao ser criada, uma instância desta classe recebe os dados pessoais do utilizador (nome, email, morada e senha), bem como o seu plano de subscrição. Com base nesse plano, o utilizador é inicialmente atribuído com um número de pontos, definidos pelo método `pontosIniciais()` do plano.

A classe também redefine o método `adicionarPontos()`, permitindo calcular dinamicamente a quantidade de pontos a adicionar, de acordo com a lógica definida no plano de subscrição, através do método `calcularPontos(Utilizador)`.

Além disso, fornece os métodos `getPlano()` e `setPlano(Subscricao)` para aceder e modificar o plano de subscrição associado ao utilizador.

Esta classe é essencial para gerir utilizadores com benefícios adicionais, personalizando o comportamento da atribuição de pontos e assegurando a integração com diferentes tipos de subscrição.

2.14 Subscrição Free

Variáveis de Instância: *(nenhuma — esta implementação não possui estado próprio; todo o comportamento é determinado pelos métodos da interface `Subscricao`)*

Descrição da Classe:
A classe `Free` concretiza o plano gratuito do serviço de streaming, implementando a interface `Subscricao`. Todas as operações que requerem funcionalidades premium devolvem `false`, reflectindo as limitações inerentes a este nível de subscrição. O método `calcularPontos()` atribui um valor fixo de 5 pontos por música, representando a bonificação mínima para utilizadores do plano gratuito.

- `getNomePlano()`: devolve a string literal `"Free"`.
- `podeCriarPlaylist()`: retorna `false`; criação de playlists pessoais não é permitida.
- `podeAvancarMusica()`: retorna `false`; salto para a próxima faixa é restrito.
- `podeRetrocederMusica()`: retorna `false`; não é possível voltar à faixa anterior.
- `podeGerarPlaylistFavoritos()`: retorna `false`; geração automática de favoritos é exclusiva de planos pagos.
- `calcularPontos(Utilizador)`: devolve `BigDecimal(5)`, pontuação básica por reprodução.

2.15 Subscrição Premium Base

Variáveis de Instância: *(nenhuma — esta implementação não armazena estado próprio; apenas define políticas via métodos da interface `Subscricao`)*

Descrição da Classe:
`PremiumBase` representa o nível de assinatura paga mais simples. Os seus titulares desfrutam de funcionalidades de controlo de reprodução e organização, mas ainda não têm acesso a recursos avançados como a geração automática de playlists de favoritos. O sistema recompensa-os com 10 pontos por reprodução.

- `getNomePlano()`: devolve `"PremiumBase"`.

- `podeCriarPlaylist(): true` — criação de playlists permitida.
- `podeAvancarMusica(): true` — salto para a faixa seguinte permitido.
- `podeRetrocederMusica(): true` — retrocesso permitido.
- `podeGerarPlaylistFavoritos(): false` — recurso reservado a níveis superiores.
- `calcularPontos(Utilizador):` devolve `BigDecimal(10)`, concedendo uma bonificação intermédia por música.

2.16 Subscrição Premium Top

Variáveis de Instância: *(nenhuma — tal como `PremiumBase`, esta classe não mantém estado próprio; estende apenas a política de métodos).*

Descrição da Classe:
`PremiumTop` é o nível de assinatura mais completo do serviço. Para além de herdar todas as permissões de `PremiumBase`, acrescenta:

- `podeGerarPlaylistFavoritos():` devolve `true`, permitindo ao assinante gerar automaticamente uma playlist com as suas músicas favoritas.
- `calcularPontos(Utilizador):` devolve pontos_actuais $\times$ 0,025, ou seja, um bónus de 2,5 % sobre o saldo de pontos do utilizador por cada reprodução, recompensando a fidelidade.
- `pontosIniciais():` método auxiliar que define 100 pontos de arranque quando o utilizador adere ao plano.

2.17 Subscrição

Variáveis de Instância: *nenhuma — por se tratar de uma `interface`, define apenas um contrato de métodos a implementar, sem estado próprio.*

Descrição da Interface:
`Subscricao` estabelece as capacidades que qualquer plano (`Free`, `PremiumBase`, `PremiumTop`, *etc.*) deve oferecer. Cada implementação decide quais funcionalidades são permitidas, bem como a política de atribuição de pontos.

- `String getNomePlano():` devolve o nome comercial do plano (por exemplo, "Free", "PremiumBase").
- `boolean podeCriarPlaylist():` indica se o utilizador pode criar playlists manuais.
- `boolean podeAvancarMusica():` indica se é permitido saltar para a faixa seguinte.
- `boolean podeRetrocederMusica():` indica se é permitido voltar à faixa anterior.

- `boolean podeGerarPlaylistFavoritos()`: indica se o plano inclui a funcionalidade de geração automática de playlists de favoritos.
- `default int pontosIniciais()`: valor base de pontos atribuídos aquando da adesão; por omissão, `0`.
- `default BigDecimal calcularPontos(Utilizador)`: calcula o bónus por reprodução; devolve `0` por defeito.

Os métodos `default` fornecem implementações-padrão, evitando código duplicado nas classes que não necessitam de lógica específica.

2.18 GestorAlbuns

Variáveis de Instância:

- **private List<Album> albuns;**
  Lista interna que armazena todos os objectos `Album` registados no sistema. Inicializada como `new ArrayList ()`.

Descrição da Classe:
`GestorAlbuns` é um *service* responsável por criar, manter e consultar álbuns de música. Fornece operações de catálogo para adicionar novos álbuns, inserir faixas, buscar informações detalhadas e produzir listagens completas.

- `void adicionarAlbum(Album album)`: insere o álbum recebido na colecção interna.
- `List<Album> listarAlbuns()`: devolve uma cópia defensiva da lista de álbuns, preservando o encapsulamento.
- `boolean existeAlbum(String nome)`: verifica, em *stream*, se há álbum com o nome dado (case-insensitive).
- `Album buscarAlbum(String nome)`: devolve o primeiro álbum cujo nome corresponde ao parâmetro ou `null` se inexistente.
- `Album criarAlbum(String nome, String artista, String editora)`: fabrica um novo `Album`; lança `IllegalArgumentException` se já existir título igual.
- `void adicionarMusicaAoAlbum(String nomeAlbum, Musica musica)`: localiza o álbum pelo nome e acrescenta-lhe a música; lança excepção se o álbum não for encontrado.
- `String obterInfoAlbum(String nome)`: devolve uma string com os metadados do álbum e a lista de músicas; indica "Álbum não encontrado." quando aplicável.
- `String listarAlbunsComMusicas()`: cria uma listagem com todos os álbuns e respectivas faixas; caso a colecção esteja vazia, devolve "Nenhum álbum registado.".

2.19 GestorEstatisticas

Variáveis de Instância:

- **private final GestorMusicas gestorMusicas;** Serviço que fornece acesso ao catálogo de músicas.
- **private final GestorPlaylists gestorPlaylists;** Responsável pela gestão de playlists (públicas e privadas).
- **private final GestorReproducao gestorReproducao;** Mantém o histórico de reproduções de músicas por utilizador e por período.
- **private final List<Utilizador> utilizadores;** Colecção de todos os utilizadores registados, utilizada para calcular estatísticas globais.

Descrição da Classe:
`GestorEstatisticas` actua como agregador de métricas do sistema. Recolhe dados de música, playlists, reproduções e utilizadores, expondo consultas de "top 10", totais, e rankings. Permite extrair rapidamente indicadores como faixas mais ouvidas, intérpretes mais populares, utilizadores mais activos e playlists com maior número de reproduções, tanto em horizonte global como limitado a um período temporal.

- `int getTotalMusicas()`: número total de músicas no catálogo.
- `int getTotalPlaylists()`: quantidade de playlists registadas.
- `int getTotalPlaylistsPublicas()`: total de playlists com visibilidade pública.
- `int getTotalUtilizadores()`: utilizadores registados.
- `List<Musica> getMusicasMaisReproduzidas()`: top 10 de faixas por reproduções globais.
- `List<Musica> getMusicasMaisReproduzidasPorPeriodo(inicio,fim)`: top 10 no intervalo indicado.
- `String getInterpreteMaisEscutado()`: artista/banda com maior número de reproduções no histórico total.
- `String getInterpreteMaisEscutadoPorPeriodo(inicio,fim)`: intérprete mais ouvido no período.
- `String getGeneroMaisReproduzido()`: género musical com mais reproduções.
- `String getUtilizadorMaisAtivo()`: endereço/email do utilizador com mais reproduções totais (via `GestorReproducao`).
- `String getUtilizadorMaisAtivoPorPeriodo(inicio,fim)`: mais ativo num período específico.

- `Utilizador getUtilizadorMaisAtivoPorHistorico()`: utilizador com maior histórico (tamanho da lista de reproduções).
- `Utilizador getUtilizadorComMaisPontos()`: utilizador com saldo de pontos mais elevado.
- `List<Map.Entry<Utilizador,Integer>> getUtilizadoresMaisAtivos()`: top 10 ordenado por número de reproduções.
- `Utilizador getUtilizadorComMaisPlaylists()`: utilizador que criou o maior número de playlists.
- `List<Playlist> getPlaylistsMaisPopulares()`: top 10 playlists por reproduções.
- `void exibirEstatisticasPorPeriodo(inicio,fim)`: imprime no console um resumo contendo:
  - top 10 músicas reproduzidas,
  - intérprete mais ouvido,
  - utilizador mais activo,

  tudo limitado ao intervalo temporal indicado.

As consultas utilizam expressões `stream` e colectores (`Collectors`) para agrupar, ordenar e limitar resultados, assegurando desempenho adequado mesmo com grandes volumes de dados.

2.20 GestorMusicas

Variáveis de Instância:

- **private List<Musica> musicas;** Lista interna que armazena todas as faixas registadas. Inicializada como `new ArrayList ()`; garante encapsulamento.

Descrição da Classe:
`GestorMusicas` é o serviço responsável por criar, manter e disponibilizar o catálogo de músicas da aplicação. Oferece operações básicas de CRUD, bem como integração com um `ImportadorJSON` para carregar registos a partir de ficheiros externos.

- `void adicionarMusica(Musica musica)`: Insere a faixa recebida na colecção.
- `List<Musica> listarMusicas()`: Devolve uma cópia defensiva da lista, evitando alterações externas.
- `boolean existeMusica(String nome)`: Verifica (ignora maiúsculas/minúsculas) se já existe música com o título indicado.

- `Musica buscarMusica(String nome):` Pesquisa e devolve a música correspondente ou `null` se não encontrada.
- `boolean importarDeJSON(String caminho):` Lê um ficheiro JSON (via `ImportadorJSON`) e adiciona ao catálogo cada faixa não repetida. Cria instâncias de `Musica` ou `MusicaExplicita`, consoante o campo `explicita` do DTO. Retorna `true` em caso de sucesso, `false` se ocorrer erro ou o ficheiro não contiver dados válidos.
- `void setMusicas(List<Musica> musicas):` Substitui a colecção interna pelo conteúdo da lista recebida (*deep copy* superficial) — útil para testes ou sincronização.

Este gestor funciona como camada de acesso central ao catálogo; outras classes (por exemplo, `GestorEstatisticas` ou `GestorPlaylists`) dependem dele para obter informações sobre faixas disponíveis.

2.21 GestorPersistencia

Variáveis de Instância:

- **private static final String DIRETORIO_DADOS;** Caminho do diretório onde os dados serão guardados (`dados/`).
- **private static final String ARQUIVO_MUSICAS;** Nome do ficheiro onde as músicas serão serializadas (`musicas.ser`).
- **private static final String ARQUIVO_PLAYLISTS;** Nome do ficheiro de playlists.
- **private static final String ARQUIVO_UTILIZADORES;** Nome do ficheiro de utilizadores.

Descrição da Classe:
`GestorPersistencia` é responsável por guardar e restaurar dados essenciais da aplicação: músicas, playlists e utilizadores. Utiliza a serialização Java para armazenar e recuperar listas de objetos. Os dados são guardados em ficheiros binários localizados no diretório `dados/`, que é criado automaticamente se não existir.

- `GestorPersistencia():` construtor que assegura a existência do diretório de dados.
- `void salvarMusicas(List<Musica>):` serializa a lista de músicas para o ficheiro `musicas.ser`.
- `void salvarPlaylists(List<Playlist>):` serializa a lista de playlists para `playlists.ser`.
- `void salvarUtilizadores(List<Utilizador>):` serializa a lista de utilizadores para `utilizadores.ser`.

- `List<Musica>carregarMusicas()`: carrega e desserializa a lista de músicas.
- `List<Playlist>carregarPlaylists()`: carrega as playlists guardadas.
- `List<Utilizador>carregarUtilizadores()`: carrega os utilizadores guardados.
- `void salvarObjeto(Object objeto, String caminhoArquivo)`: método auxiliar privado que escreve o objeto num ficheiro.
- `Object carregarObjeto(String caminhoArquivo)`: método auxiliar que tenta ler um objeto de ficheiro; retorna uma lista vazia em caso de falha.

A classe foi concebida para ser reutilizável e robusta. Em caso de erro de I/O ou ausência de ficheiro, o sistema regista a ocorrência e continua a execução, evitando falhas fatais.

Notas Adicionais:

- Os métodos de `carregar*` usam `@SuppressWarnings("unchecked")` pois envolvem cast de objetos genéricos após desserialização.
- Em caso de ausência de ficheiro, retorna-se uma `ArrayList` vazia, garantindo segurança contra `null`.
- O diretório de dados é criado apenas uma vez no construtor, o que evita redundância.

2.22 GestorPlaylists

Variáveis de Instância:

- **private List<Playlist> playlists;** Lista de todas as playlists existentes no sistema (públicas ou privadas).
- **private final GestorMusicas gestorMusicas;** Dependência utilizada para associar músicas às playlists, especialmente durante importações.

Descrição da Classe:
`GestorPlaylists` é o serviço responsável por gerir a criação, pesquisa, importação e organização de playlists. É compatível com vários tipos de playlists, como personalizadas, aleatórias e por género/duração. Permite ainda consultar informações detalhadas e listar playlists públicas ou de um determinado utilizador.

- `void adicionarPlaylist(Playlist)`: adiciona uma nova playlist à lista interna.
- `List<Playlist> listarPlaylists()`: devolve uma cópia das playlists registadas.

- `boolean existePlaylist(String nome)`: verifica se já existe uma playlist com o nome fornecido (ignorando maiúsculas/minúsculas).
- `Playlist buscarPlaylist(String nome)`: retorna a playlist com o nome indicado, ou `null` se não existir.
- `boolean importarDeJSON(String caminho)`: importa playlists a partir de um ficheiro JSON com auxílio do `ImportadorJSON`. Cada entrada origina uma `PlaylistPersonalizada`, sendo adicionadas apenas músicas que já existam no sistema.
- `Playlist criarPlaylistPersonalizada(String nome, boolean publica, boolean aleatoria, String emailCriador)`: cria uma nova `PlaylistPersonalizada` com os atributos fornecidos e regista o criador.
- `Playlist criarPlaylistAleatoria(String nome, String emailCriador)`: cria uma playlist do tipo `PlaylistAleatoria`.
- `Playlist criarPlaylistPorGeneroEDuracao(String nome, List<Musica>, String genero, int maxSegundos, String emailCriador)`: gera uma playlist baseada num género específico e com duração total limitada.
- `void adicionarMusicaAPlaylist(String nomePlaylist, Musica)`: adiciona uma música à playlist identificada. Lança exceção se a playlist não for encontrada.
- `String obterInfoPlaylist(String nome)`: devolve uma descrição textual detalhada de uma playlist, incluindo nome, tipo, visibilidade, número de músicas, duração e lista de faixas.
- `String listarPlaylistsComMusicas()`: devolve informações detalhadas sobre todas as playlists presentes no sistema. Retorna aviso caso não exista nenhuma.
- `List<Playlist> listarPlaylistsPublicas()`: retorna apenas as playlists visíveis a todos os utilizadores.
- `List<Playlist> listarPlaylistsDoUtilizador(Utilizador)`: retorna todas as playlists criadas por um determinado utilizador (usando o email como identificador).
- `void setPlaylists(List<Playlist> playlists)`: substitui a lista interna por uma nova, útil para carregamentos externos ou redefinições.

Este gestor promove a centralização das operações sobre playlists e facilita a integração com outras partes do sistema, como o gestor de músicas ou o módulo de persistência. Também suporta importação de dados externos, permitindo configuração inicial ou migração.

2.23 GestorRecomendacoes

Variáveis de Instância:

- **private final List<Musica> musicas;** Lista de músicas disponíveis no sistema.
- **private final List<Playlist> playlists;** Lista de playlists disponíveis para recomendação.
- **private final GestorMusicas gestorMusicas;** Referência ao gestor de músicas, usada para obter o catálogo completo em filtros por género ou artista.

Descrição da Classe:
`GestorRecomendacoes` centraliza a lógica de sugestão de conteúdos para utilizadores com base no seu histórico de audições. Oferece recomendações personalizadas de músicas e playlists, organizadas por género, artista e popularidade.

- `List<Musica> recomendarMusicas(Utilizador):` Recomenda até 10 músicas que o utilizador ainda não ouviu, priorizando géneros que mais consome.
- `List<Playlist> recomendarPlaylists(Utilizador):` Sugere até 5 playlists que contenham músicas dos géneros favoritos do utilizador, ordenadas pela quantidade dessas músicas.
- `String getRecomendacoesUsuario(Utilizador):` Consolida as recomendações de músicas e playlists numa mensagem legível, adequada para exibição textual.
- `List<Musica> getRecomendacoesPorGenero(String, Utilizador):` Sugere até 10 músicas do género indicado que ainda não tenham sido reproduzidas pelo utilizador, priorizando as mais populares.
- `List<Musica> getRecomendacoesPorArtista(String, Utilizador):` Lista até 10 faixas do artista especificado que o utilizador nunca ouviu, ordenadas por número de reproduções.
- `List<Musica> getRecomendacoesPorPopularidade(Utilizador):` Apresenta as 10 músicas mais reproduzidas da plataforma que o utilizador ainda não escutou.

Este gestor aplica uma abordagem baseada em preferências implícitas — ou seja, analisa o histórico de reproduções para inferir interesses. Além disso, evita repetições ao garantir que apenas músicas ainda não ouvidas sejam sugeridas. Com isso, melhora a experiência de utilizador, promovendo descoberta de novas faixas e playlists com base no seu perfil de escuta.

2.24 GestorReproducao

Variáveis de Instância:

- **private final List<Musica> filaReproducao;** Lista de músicas em reprodução contínua (fila actual).
- **private final List<Reproducao> historicoReproducoes;** Registo completo de todas as reproduções feitas no sistema, com data e utilizador associado.
- **private int indiceAtual;** Índice da música actualmente em reprodução na fila.
- **private boolean reproduzindo;** Indica se a reprodução está activa (true) ou pausada/parada (false).
- **private Utilizador utilizadorAtual;** Referência ao utilizador que iniciou a reprodução actual (necessária para registo no histórico).

Descrição da Classe:
`GestorReproducao` é responsável pela simulação de um player musical com suporte a playlists, controlo de faixas, e registo do histórico de reprodução. Controla o fluxo de reprodução de músicas e playlists, associando cada reprodução a um utilizador específico. É também capaz de determinar estatísticas de utilização com base em datas.

- `void setUtilizadorAtual(Utilizador):` actualiza o utilizador responsável pelas próximas reproduções.
- `void reproduzirMusica(Musica, Utilizador):` inicia a reprodução de uma única música, limpando a fila anterior. Regista a reprodução no histórico, associada ao utilizador.
- `void reproduzirPlaylist(Playlist, Utilizador):` inicia a reprodução sequencial de uma playlist. Todas as músicas são registadas no histórico.
- `void pausarRetomar():` alterna entre estados de reprodução e pausa.
- `void parar():` pára completamente a reprodução e reinicia o estado da fila.
- `void proximaMusica():` avança para a música seguinte na fila, se disponível.
- `void musicaAnterior():` retrocede para a música anterior, se possível.
- `boolean estaReproduzindo():` indica se há reprodução em curso.
- `Musica getMusicaAtual():` retorna a música actualmente em reprodução, ou `null` se não houver.

- `List<Musica> getFilaReproducao()`: devolve a fila de músicas como cópia defensiva.
- `List<Reproducao> getHistoricoReproducoes()`: histórico completo de reproduções realizadas.
- `List<Reproducao> getHistoricoReproducoesPorPeriodo(inicio, fim)`: filtra o histórico com base num intervalo de tempo.
- `String getUtilizadorMaisAtivo(inicio, fim)`: calcula qual utilizador reproduziu mais músicas dentro de um determinado período.

Esta classe integra funcionalidades essenciais tanto para reprodução interactiva como para análise de comportamento de utilizadores. Serve de base para módulos como estatísticas e recomendações, garantindo registo fiel das interacções com o sistema.

2.25 GestorUtilizadores

Variáveis de Instância:

- **private final List<Utilizador> utilizadores;** Lista de todos os utilizadores registados na aplicação, incluindo utilizadores ocasionais e premium.

Descrição da Classe:
A classe `GestorUtilizadores` gere toda a lógica relacionada com o registo, autenticação, importação, consulta e actualização de utilizadores. É responsável por assegurar que cada utilizador seja único, que os planos de subscrição estejam actualizados e que os dados possam ser importados de fontes externas como ficheiros JSON.

- `boolean importarDeJSON(String caminho)`: Carrega dados de utilizadores a partir de um ficheiro JSON e cria instâncias de `UtilizadorOcasional` ou `UtilizadorPremium` com base no plano especificado. Utiliza `ImportadorJSON` e evita duplicações.
- `boolean existeUtilizador(String nome)`: Verifica se já existe um utilizador com o nome fornecido.
- `List<Utilizador> listarUtilizadores()`: Devolve uma cópia da lista de utilizadores registados.
- `Utilizador buscarUtilizador(String email)`: Pesquisa um utilizador pelo endereço de e-mail (ignora maiúsculas/minúsculas).
- `Utilizador fazerLogin(String email, String senha)`: Verifica se existe um utilizador com o e-mail e senha fornecidos. Retorna a instância correspondente em caso de sucesso.
- `Utilizador registar(String nome, String senha, String email)`: Cria um novo `UtilizadorOcasional`, se o nome ainda não estiver em uso.

- `boolean atualizarUtilizador(String email, String novoEmail, String novaSenha)`: Actualiza os dados de contacto e autenticação de um utilizador. Retorna `false` se o utilizador não for encontrado.
- `boolean removerUtilizador(String email)`: Remove um utilizador da lista se existir.
- `boolean mudarPlano(String email, String tipoPlano)`: Permite migrar um utilizador entre os planos `Free`, `PremiumBase` e `PremiumTop`. Preserva os dados existentes e apenas actualiza o tipo de subscrição. Utilizadores que já se encontram no plano desejado não são modificados.

Observações:

- Ao mudar o plano para `Free`, o utilizador premium é convertido numa instância de `UtilizadorOcasional`.
- O sistema garante consistência ao substituir o objecto na lista por um novo com os mesmos dados básicos.
- Utiliza 'instanceof' para verificar o tipo actual do utilizador antes de efectuar modificações no plano.

Esta classe é essencial para o funcionamento do sistema de autenticação, personalização e gestão de acesso baseado em planos.

2.26 AlbumUI

Variáveis de Instância:

- **private final GestorAlbuns gestorAlbuns;** Serviço responsável pela criação, consulta e gestão de álbuns.
- **private final GestorMusicas gestorMusicas;** Serviço auxiliar usado para pesquisar músicas ao adicionar ou remover faixas de um álbum.

Descrição da Classe:
`AlbumUI` é uma interface de consola que permite a gestão de álbuns musicais. Herda de `MenuBase` e apresenta um menu interactivo com várias opções de acção. Facilita a interação com as funcionalidades principais do `GestorAlbuns`, como criação, listagem e edição de conteúdos.

Funcionalidades do Menu:

- `1. Criar Álbum` — Permite criar um novo álbum, solicitando nome, artista e editora. Verifica se já existe um álbum com o mesmo nome.
- `2. Listar Álbuns` — Apresenta uma listagem completa dos álbuns registados, com detalhes como número de músicas e duração total.
- `3. Adicionar Música ao Álbum` — Solicita o nome de um álbum e de uma música. Se ambos existirem, a música é adicionada ao álbum.

- `4. Remover Música do Álbum` — Permite remover uma música específica de um álbum existente.
- `5. Voltar` — Retorna ao menu anterior.

Métodos Auxiliares:

- `mostrarOpcoes()`: lista as opções disponíveis ao utilizador.
- `processarOpcao(int)`: executa a acção correspondente à opção seleccionada.
- `deveSair(int)`: retorna `true` se o utilizador escolheu a opção de voltar ao menu principal.
- `criarAlbum()`: interage com o utilizador para obter os dados de um novo álbum e adicioná-lo via `gestorAlbuns`.
- `listarAlbuns()`: utiliza o `gestorAlbuns` para obter e formatar a lista de álbuns, exibindo os seus dados e músicas.
- `adicionarMusica()`: procura um álbum e uma música pelo nome e, se ambos forem válidos, associa a música ao álbum.
- `removerMusica()`: procura o álbum e a música, removendo a música da lista interna do álbum caso exista.

Observações:

- A leitura de texto é feita com apoio de `InputUtil.lerTexto()`, promovendo reutilização e validação centralizada.
- Utiliza o padrão de herança de menus (`MenuBase`) para uniformizar a navegação por consola.
- Todas as interações são feitas via consola (`Scanner`) com resposta textual ao utilizador.

2.27 ReproducaoUI

Variáveis de Instância:

- **private final GestorReproducao gestorReproducao;** Responsável pela lógica de reprodução de músicas e playlists.
- **private final GestorMusicas gestorMusicas;** Permite acesso e pesquisa de músicas disponíveis.
- **private final GestorPlaylists gestorPlaylists;** Utilizado para buscar playlists existentes.
- **private final PlayerUI playerUI;** Interface responsável pela simulação/interação da reprodução de músicas.

- **private Utilizador utilizadorAtual;** Armazena o utilizador atualmente autenticado que está a utilizar a reprodução.

A classe `ReproducaoUI` define a interface de utilizador relacionada à reprodução de músicas ou playlists. Herda de `MenuBase` para reutilização de lógica comum de menus em consola. Métodos principais:

- **mostrarOpcoes():** Apresenta ao utilizador as opções disponíveis: reproduzir música, reproduzir playlist ou voltar ao menu anterior.
- **processarOpcao(int opcao):** Executa a ação escolhida com base na opção selecionada pelo utilizador.
- **deveSair(int opcao):** Indica se o menu deve ser encerrado (opção 3 – Voltar).
- **setUtilizadorAtual(Utilizador utilizador):** Define o utilizador ativo que está a utilizar o sistema no momento.
- **reproduzirMusica():** Solicita o nome da música, procura no sistema e, se encontrada, executa a reprodução para o utilizador atual.
- **reproduzirPlaylist():** Solicita o nome da playlist, procura no sistema e, se encontrada, executa a reprodução da playlist para o utilizador atual, iniciando pela primeira música da lista (caso não esteja vazia).

Esta classe é essencial para a funcionalidade de reprodução de conteúdo musical na aplicação, conectando a lógica de negócio com a interação direta com o utilizador. Ela encapsula toda a experiência de audição via consola, respeitando o contexto do utilizador autenticado.

2.28 BibliotecaUI

Variáveis de Instância:

- **private final GestorMusicas gestorMusicas;** Serviço responsável por fornecer acesso à lista de músicas registadas no sistema.

Descrição da Classe:
`BibliotecaUI` fornece uma interface de consola que permite ao utilizador consultar a biblioteca musical. Herda de `MenuBase` e apresenta um menu simples com opções para listar músicas ou procurar uma faixa específica.

Funcionalidades do Menu:

- `1. Listar Músicas` — Apresenta uma lista de todas as músicas disponíveis, mostrando o nome e o intérprete de cada uma.
- `2. Buscar Música` — Solicita ao utilizador o nome de uma música e, caso exista, exibe os seus dados detalhados: nome, intérprete, duração e género.

- `3. Voltar` — Retorna ao menu principal.

Métodos Auxiliares:

- `mostrarOpcoes()`: exibe as opções disponíveis ao utilizador.
- `processarOpcao(int)`: executa a acção correspondente à opção seleccionada.
- `deveSair(int)`: indica se o utilizador optou por sair do menu.
- `listarMusicas()`: utiliza o `gestorMusicas` para recuperar todas as músicas registadas e formata-as para exibição. Caso não existam músicas, apresenta uma mensagem informativa.
- `buscarMusica()`: pede ao utilizador o nome de uma música, pesquisa-a através do `gestorMusicas`, e exibe os detalhes se encontrada.

Observações:

- Toda a leitura de dados é feita por consola com o apoio da classe utilitária `InputUtil`.
- A classe segue o padrão comum de UI no sistema, com navegação por opções numéricas e estrutura modular.
- A exibição dos resultados é centralizada em `mostrarResultado()`, herdado de `MenuBase`.

2.29 EstadoUI

Variáveis de Instância:

- **private final GestorPersistencia gestorPersistencia;** Serviço responsável por guardar e carregar dados do sistema em ficheiros serializados.
- **private final GestorMusicas gestorMusicas;** Serviço que gere o catálogo de músicas da aplicação.
- **private final GestorPlaylists gestorPlaylists;** Responsável pela gestão das playlists registadas.
- **private final List<Utilizador> utilizadores;** Lista partilhada de utilizadores actualmente activos no sistema.

Descrição da Classe:
`EstadoUI` fornece uma interface de linha de comandos que permite ao utilizador guardar e restaurar o estado completo da aplicação. Esta interface actua como intermediária entre o utilizador e o `GestorPersistencia`, oferecendo controlo manual sobre a persistência dos dados em disco.

Funcionalidades do Menu:

- `1. Guardar estado` — Serializa os dados actuais (músicas, playlists, utilizadores) para ficheiros no sistema. Caso a operação seja bem-sucedida, informa o utilizador.
- `2. Carregar estado` — Recupera os dados gravados previamente, substituindo os actuais. Apenas efectua substituição se os ficheiros contiverem dados válidos.
- `0. Voltar` — Termina a sessão desta interface e retorna ao menu anterior.

Métodos Auxiliares:

- `mostrarOpcoes()`: Apresenta as opções disponíveis no menu de gestão de estado.
- `processarOpcao(int)`: Executa a operação correspondente à opção seleccionada (guardar ou carregar).
- `deveSair(int)`: Indica se o menu deve ser encerrado com base na opção seleccionada.
- `guardarEstado()`: Recolhe os dados actuais e envia-os ao `GestorPersistencia` para escrita em ficheiro. Captura possíveis excepções de I/O e informa o utilizador do sucesso ou falha da operação.
- `carregarEstado()`: Lê os dados persistidos e substitui as listas actuais do sistema pelas versões recuperadas. Apenas realiza actualizações se os dados carregados forem não vazios.

Observações:

- Esta interface simplifica o processo de backup e restauro de sessões para o utilizador.
- O uso de uma lista de utilizadores partilhada permite sincronizar dados entre diferentes componentes do sistema.
- As operações são protegidas contra erros, assegurando robustez mesmo em caso de ficheiros corrompidos ou inexistentes.

2.30 EstatisticasUI

Variáveis de Instância:

- **private final GestorEstatisticas gestorEstatisticas;** Serviço responsável por calcular e fornecer dados estatísticos globais e por período.
- **private static final DateTimeFormatter formatter;** Formato utilizado para leitura de datas introduzidas pelo utilizador na forma `dd/MM/yyyy HH:mm`.

Descrição da Classe:
`EstatisticasUI` disponibiliza uma interface textual que permite consultar estatísticas gerais e por intervalo de tempo, com base nos dados fornecidos pelo `GestorEstatisticas`. É uma ferramenta de análise que fornece rapidamente uma visão sobre as músicas, intérpretes e utilizadores mais activos na plataforma.

Funcionalidades do Menu:

- `1. Ver Estatísticas Gerais` — Apresenta o número total de músicas, as 10 mais reproduzidas, o intérprete mais escutado e o utilizador mais activo.
- `2. Ver Estatísticas por Período` — Solicita ao utilizador um intervalo de datas e exibe as estatísticas apenas para esse período. Os dados incluem músicas mais reproduzidas, intérprete mais ouvido e utilizador mais activo.
- `3. Voltar` — Retorna ao menu principal.

Métodos Auxiliares:

- `mostrarOpcoes()`: Apresenta as opções disponíveis ao utilizador.
- `processarOpcao(int)`: Executa a acção correspondente à opção seleccionada.
- `deveSair(int)`: Determina se o utilizador deseja sair da interface.
- `exibirEstatisticasGerais()`: Constrói e exibe um resumo textual das estatísticas globais recolhidas do sistema.
- `exibirEstatisticasPorPeriodo()`: Solicita datas de início e fim ao utilizador e invoca o `gestorEstatisticas` para calcular os dados correspondentes ao intervalo.
- `lerDataHora(String)`: Método robusto de leitura de datas, validando o formato introduzido e repetindo a leitura em caso de erro.

Observações:

- A classe utiliza `InputUtil` para facilitar a leitura de texto.
- As datas são interpretadas com `DateTimeFormatter` para garantir consistência e compatibilidade com o histórico de reproduções.
- Este menu foca-se exclusivamente em apresentação de dados, sem modificar o estado do sistema.

2.31 LoginUI

Variáveis de Instância:

- **private final GestorUtilizadores gestorUtilizadores;** Responsável pela autenticação, registo e gestão de utilizadores.
- **private final GestorPlaylists gestorPlaylists;** Usado para interagir com as playlists durante a sessão do utilizador.
- **private final GestorMusicas gestorMusicas;** Permite acesso à biblioteca de músicas para navegação ou reprodução.
- **private final GestorRecomendacoes gestorRecomendacoes;** Fornece sugestões personalizadas com base no histórico do utilizador.
- **private final GestorReproducao gestorReproducao;** Controla a reprodução de músicas e playlists.
- **private final GestorEstatisticas gestorEstatisticas;** Gera dados analíticos sobre músicas, utilizadores e interacções.
- **private Utilizador utilizadorAtual;** Utilizador autenticado na sessão actual.

Descrição da Classe:
`LoginUI` é a interface de entrada do utilizador no sistema. Permite efectuar login, registar novos utilizadores e aceder ao menu pessoal com diversas funcionalidades, desde biblioteca até estatísticas.

Funcionalidades do Menu Principal:

- `1. Login` — Solicita e-mail e senha. Se forem válidos, abre o menu pessoal do utilizador.
- `2. Registar` — Solicita nome, e-mail e senha para criar um novo `UtilizadorOcasional`.
- `3. Voltar` — Retorna ao menu anterior sem iniciar sessão.

Menu do Utilizador (pós-login):

- `1. Biblioteca` — Acede ao menu `BibliotecaUI` para listar e consultar músicas.
- `2. Playlists` — Abre o menu `PlaylistUI` para criação e gestão de playlists pessoais.
- `3. Reproduzir` — Permite iniciar a reprodução de músicas e playlists via `ReproducaoUI`.
- `4. Ver Recomendações` — Mostra sugestões musicais personalizadas com base no histórico do utilizador.
- `5. Ver Estatísticas Pessoais` — Exibe estatísticas relevantes da actividade do utilizador através de `Estatisticas`.

2.32 Menu

Descrição da Classe:
A classe `Menu` contém o ponto de entrada (`main`) da aplicação e coordena a inicialização de todos os componentes principais do sistema. É responsável por preparar os gestores de dados, carregar os ficheiros persistidos (caso existam), lançar o menu principal e garantir a gravação final do estado ao sair.

Responsabilidades principais:

- Criar instâncias dos gestores:
  - `GestorMusicas, GestorAlbuns, GestorPlaylists, GestorReproducao,`
  - `GestorUtilizadores, GestorEstatisticas, GestorRecomendacoes, GestorPersistencia`.
- Criar e validar o diretório `dados/` onde os ficheiros de persistência são armazenados (`musicas.ser`, `playlists.ser`, `utilizadores.ser`).
- Carregar os dados previamente guardados, se existirem, para as estruturas internas dos gestores correspondentes.
- Inicializar interfaces principais:
  - `LoginUI` — autenticação e navegação pessoal do utilizador.
  - `EstadoUI` — gestão manual de carregamento/armazenamento de dados.
  - `MenuPrincipal` — menu geral da aplicação, acessível a todos os utilizadores.
- Invocar `menuPrincipal.mostrarMenu()` para iniciar o ciclo principal da aplicação.
- Salvar os dados (músicas, playlists e utilizadores) no final da execução.

Observações:

- O sistema assegura que dados em memória não se perdem ao terminar o programa, graças à utilização de `GestorPersistencia`.
- As dependências são injectadas explicitamente nas interfaces de utilizador, promovendo modularidade e teste.
- Os dados carregados são validados para evitar substituição desnecessária por listas vazias.

Fluxo resumido:

1. Inicialização dos gestores e interfaces.
2. Carregamento de dados persistidos.
3. Execução do menu principal.
4. Gravação dos dados ao terminar.

2.33 MenuBase

Variáveis de Instância:

- **protected final Scanner scanner;** Instância partilhada para leitura de dados da linha de comandos.
- **protected final String titulo;** Título do menu, utilizado como cabeçalho nas interacções com o utilizador.

Descrição da Classe:
`MenuBase` é uma classe abstracta que define a estrutura fundamental para todos os menus da aplicação. Centraliza o ciclo de exibição, leitura de opções e navegação, promovendo reutilização de código e consistência visual nas interfaces textuais.

Métodos Principais:

- `MenuBase(Scanner scanner, String titulo):` Construtor que inicializa o leitor da consola e o título do menu.
- `void mostrarMenu():` Inicia o ciclo de execução do menu. Enquanto o utilizador não seleccionar a opção de saída, o menu:
  1. limpa o conteúdo do ecrã até ao fim;
  2. exibe o cabeçalho;
  3. apresenta as opções disponíveis;
  4. lê a opção introduzida;
  5. processa a acção correspondente;
  6. verifica se deve terminar.
- `protected void exibirCabecalho():` Exibe o título do menu com uma formatação padronizada.
- `protected void mostrarResultado(String mensagem):` Limpa o ecrã, apresenta o resultado da operação e aguarda que o utilizador pressione ENTER para continuar.
- `protected abstract void mostrarOpcoes():` Método abstracto que deve ser implementado pelas subclasses para apresentar as opções específicas do menu.
- `protected abstract void processarOpcao(int opcao):` Define a acção a executar com base na opção seleccionada pelo utilizador.
- `protected abstract boolean deveSair(int opcao):` Indica se a execução do menu deve ser encerrada com base na opção introduzida.

Observações:

- Esta classe serve como base para todas as interfaces com menus interactivos no sistema.
- A limpeza do terminal é feita através da classe auxiliar `ConsoleUtil`.
- Promove separação entre lógica de apresentação e lógica de negócio, permitindo reutilização e organização modular.

2.34 MenuPrincipal

Variáveis de Instância:

- **private final GestorMusicas gestorMusicas;** Serviço responsável pela gestão do catálogo de músicas.
- **private final GestorPlaylists gestorPlaylists;** Responsável pelas operações sobre playlists, incluindo criação, listagem e importação.
- **private final GestorEstatisticas gestorEstatisticas;** Fornece estatísticas globais e por período sobre utilização, músicas e utilizadores.
- **private final LoginUI loginUI;** Interface responsável pelo login, registo e navegação pessoal dos utilizadores.
- **private final EstadoUI estadoUI;** Interface que permite guardar e carregar o estado persistente da aplicação.

Descrição da Classe:
A classe `MenuPrincipal` representa o ponto central de navegação da aplicação. É a interface inicial com o utilizador e oferece acesso a todas as funcionalidades principais: gestão de conteúdos, estatísticas, autenticação e persistência.

Funcionalidades do Menu Principal:

- `1. Gestão` — Acede a um sub-menu com opções para gerir músicas, playlists, utilizadores e estado.
- `2. Estatísticas` — Permite consultar estatísticas gerais ou por período.
- `3. Login/Registo` — Abre a interface de autenticação de utilizadores.
- `4. Sair` — Encerra a execução da aplicação.

Sub-menus e Navegação:

- `mostrarMenuGestao()`: Apresenta um menu com:
  - Gestão de Músicas (`MusicaUI`)
  - Gestão de Playlists (`PlaylistUI`)
  - Gestão de Utilizadores (`UtilizadorUI`)
  - Gestão de Estado (`EstadoUI`)

- `mostrarMenuEstatisticas()`: Permite ver estatísticas através de `EstatisticasUI`.
- `mostrarMenuMusicas()`: Lança a interface de gestão musical.
- `mostrarMenuPlaylists()`: Lança a interface de gestão de playlists.
- `mostrarMenuUtilizadores()`: Acede à gestão directa dos utilizadores registados.
- `mostrarMenuLogin()`: Encaminha o utilizador para o processo de login ou registo.

Observações:

- O menu principal funciona com base na estrutura definida em `MenuBase`.
- A navegação entre menus é modular, com cada funcionalidade encapsulada na sua própria classe de interface.
- A limpeza do terminal é feita através da utilidade `ConsoleUtil` para garantir legibilidade.
- O menu adapta-se ao estado actual, permitindo acesso personalizado conforme o utilizador esteja autenticado ou não.

2.35 MusicaUI

Variáveis de Instância:

- **private final GestorMusicas gestorMusicas;** Responsável pela lógica de gestão das músicas, como criação, listagem e importação de dados.

A classe `MusicaUI` representa o menu de gestão de músicas no sistema, disponibilizando opções que permitem aos utilizadores criar, listar e importar músicas.

Ao iniciar, o menu apresenta as seguintes opções:

- **1. Criar Música** — Permite ao utilizador inserir os dados de uma nova música, incluindo nome, intérprete, género, duração, letra, editora e se é explícita. Se for explícita, a instância criada será de `MusicaExplicita`.
- **2. Listar Músicas** — Mostra todas as músicas registadas no sistema com os seus principais atributos (nome, intérprete, género, duração e número de reproduções).
- **3. Importar de JSON** — Importa músicas a partir de um ficheiro JSON localizado em `src/main/resources/dados.json`.
- **4. Voltar** — Retorna ao menu principal.

O processo de criação de música é interativo e realiza validações para garantir:

- Que o nome da música não exista já no sistema.
- Que a duração da música seja um número positivo.
- Que a resposta à pergunta sobre se a música é explícita seja válida.

A classe faz uso do `InputUtil` para facilitar a recolha e validação de dados inseridos pelo utilizador. Ao final de cada operação, é apresentado um resumo ou feedback textual para informar o utilizador do sucesso ou falha da ação executada.

2.36 PlayerUI

Variáveis de Instância:

- **private final GestorReproducao gestorReproducao;** Responsável por gerir a lógica de navegação entre músicas (como avançar ou retroceder na fila de reprodução).
- **private final Scanner scanner;** Permite ler os comandos inseridos pelo utilizador durante a reprodução da música.
- **private ScheduledExecutorService executor;** Utilizado para agendar a exibição linha a linha da letra da música, simulando a sua reprodução ao longo do tempo.
- **private boolean reproduzindo;** Indica se a música está atualmente a ser reproduzida. É usado para controlar o ciclo de reprodução no interface.
- **private int linhaAtual;** Representa o índice da linha atual da letra que está a ser exibida. É incrementado à medida que a música "avança".
- **private List<String> letras;** Armazena as linhas da letra da música, obtidas ao dividir o texto completo da letra por quebras de linha.

A classe `PlayerUI` tem como objetivo fornecer uma interface de reprodução de música em modo consola. Esta interface simula uma experiência de reprodução contínua ao apresentar a letra da música linha por linha em intervalos regulares de tempo. Durante a reprodução, o utilizador pode interagir em tempo real através de comandos simples:

- **p** — Pausar ou retomar a reprodução.
- **n** — Avançar para a próxima música.
- **a** — Recuar para a música anterior.
- **q** — Terminar a reprodução.

A lógica de reprodução é executada com recurso a um `ScheduledExecutorService`, que controla a exibição gradual das linhas da letra. A classe colabora com o `GestorReproducao` para garantir uma transição adequada entre músicas. Esta interface torna a reprodução textual mais envolvente e oferece uma experiência interativa para o utilizador mesmo num ambiente de consola.

2.37 PlaylistUI

Variáveis de Instância:

- **private final GestorPlaylists gestorPlaylists;** Responsável pela gestão de playlists. Permite criar, buscar e listar playlists, bem como realizar operações sobre elas.
- **private final GestorMusicas gestorMusicas;** Fornece acesso às músicas disponíveis no sistema, sendo utilizado para adicionar ou remover músicas de playlists.
- **private Utilizador utilizadorAtual;** Representa o utilizador atualmente autenticado. Dependendo do tipo de utilizador (normal ou premium), disponibiliza diferentes funcionalidades.

A classe `PlaylistUI` implementa uma interface de consola que permite ao utilizador interagir com funcionalidades relacionadas com a criação e gestão de playlists. Estende a classe `MenuBase`, oferecendo um menu dinâmico cujas opções variam conforme o tipo de utilizador:

- Utilizadores não autenticados ou normais têm acesso apenas à listagem de playlists públicas.
- Utilizadores premium têm acesso a funcionalidades adicionais como criar playlists, adicionar e remover músicas, e (futuramente) gerar playlists automáticas com base em preferências e critérios personalizados.

A interface recolhe dados do utilizador através de entradas na consola e utiliza os serviços de `GestorPlaylists` e `GestorMusicas` para efetuar operações no sistema. Os resultados são apresentados de forma formatada para facilitar a visualização e navegação. Esta classe é essencial para a interação do utilizador com o sistema de playlists, permitindo personalizar e gerir coleções musicais de forma intuitiva.

2.38 RecomendacoesUI

Variáveis de Instância:

- **private final GestorRecomendacoes gestorRecomendacoes;** Responsável por fornecer as recomendações musicais. Atua como ligação entre a interface de utilizador e a lógica de recomendação do sistema.
- **private Utilizador utilizadorAtual;** Representa o utilizador que está atualmente a interagir com a interface de recomendações. Este utilizador é usado para personalizar as recomendações geradas.

A classe `RecomendacoesUI` tem como objetivo apresentar ao utilizador um menu interativo com recomendações musicais personalizadas. Esta classe estende `MenuBase`, permitindo que o utilizador visualize músicas recomendadas com base em três critérios distintos: género musical, artista ou popularidade. As

interações são feitas por meio da consola, utilizando um `Scanner` para ler as opções e preferências do utilizador. Através dos métodos fornecidos por `GestorRecomendacoes`, a classe obtém as listas de músicas recomendadas e apresenta-as de forma clara, facilitando a navegação e descoberta musical. Esta interface é essencial para tornar o sistema mais dinâmico e adaptado às preferências dos utilizadores, promovendo a exploração de novos conteúdos musicais.

2.39 UtilizadorUI

Variáveis de Instância:

- **private final GestorUtilizadores gestorUtilizadores;** Responsável pela gestão das operações relacionadas aos utilizadores, como listagem, edição, remoção e importação.

A classe `UtilizadorUI` é responsável por apresentar e gerir o menu de interface do utilizador para funcionalidades relacionadas à gestão de utilizadores na aplicação. Herda de `MenuBase`, aproveitando funcionalidades comuns de menus baseados em consola. Métodos principais:

- **mostrarOpcoes():** Exibe as opções disponíveis no menu da gestão de utilizadores.
- **processarOpcao(int opcao):** Executa a ação correspondente à opção escolhida pelo utilizador, como listar utilizadores, editar dados, ou importar de ficheiro JSON.
- **deveSair(int opcao):** Determina se o menu deve encerrar com base na opção fornecida (neste caso, opção 7 para voltar).
- **listarUtilizadores():** Apresenta na consola todos os utilizadores registados, com nome e email.
- **verDetalhesUtilizador():** Mostra informações detalhadas de um utilizador específico, incluindo o plano de subscrição, caso aplicável.
- **editarUtilizador():** Permite modificar o email e a senha de um utilizador existente.
- **removerUtilizador():** Remove um utilizador do sistema com base no seu email.
- **mudarPlano():** Permite alterar o plano de subscrição de um utilizador, oferecendo opções entre "Free", "Premium Base"e "Premium Top".
- **importarDeJSON():** Realiza a importação de utilizadores a partir de um ficheiro JSON pré-definido.

Esta classe centraliza todas as interações de consola relacionadas com utilizadores, facilitando o acesso às operações administrativas do sistema e promovendo a separação de responsabilidades entre lógica de aplicação e apresentação.

2.40 ConsoleUtil

Variáveis de Instância:

- **(nenhuma)** Esta classe não possui variáveis de instância. Todos os seus métodos são estáticos e utilitários, voltados para manipulação do terminal.

A classe `ConsoleUtil` fornece um conjunto de métodos estáticos para interagir e manipular a saída do terminal de forma a melhorar a apresentação da interface textual. Esta classe é especialmente útil em aplicações de linha de comando que exigem uma interface mais dinâmica e limpa.

- `public static void limparTela()` Limpa completamente o terminal. Tenta usar o comando nativo do sistema operativo para limpar a tela. Caso não seja possível, utiliza uma alternativa simples imprimindo múltiplas linhas em branco.
- `public static void limparLinha()` Remove o conteúdo da linha atual no terminal, movendo o cursor para o início da linha e apagando o texto.
- `public static void limparAteFim()` Limpa do cursor até o final da tela, útil para atualizar parcialmente o conteúdo do terminal.
- `public static void limparAteFimLinha()` Limpa apenas do cursor até o fim da linha atual.
- `public static void sobrescreverLinha(String texto)` Apaga o conteúdo da linha atual e escreve um novo texto no seu lugar. Ideal para atualizar status ou mensagens sem criar novas linhas.
- `public static void atualizarLinha(String texto)` Funciona de forma semelhante ao método anterior, sendo útil para manter o terminal limpo durante atualizações de conteúdo.
- `public static void moverCursor(int linhas)` Move o cursor verticalmente no terminal. Um valor positivo move para cima e um valor negativo move para baixo.

A `ConsoleUtil` é fundamental para a criação de interfaces mais interativas em aplicações de terminal, melhorando significativamente a experiência do utilizador ao permitir atualizações visuais dinâmicas sem poluir a saída com múltiplas linhas.

2.41 DadosImportacao

Variáveis de Instância:

- **public List<UtilizadorDTO> utilizadores;** Lista de objetos do tipo `UtilizadorDTO`, representando os utilizadores a serem importados para o sistema.
- **public List<MusicaDTO> musicas;** Lista de músicas a serem importadas, representadas por instâncias da classe `MusicaDTO`.

- **public List<PlaylistDTO> playlists;** Lista de playlists a serem importadas, cada uma representada por um objeto do tipo `PlaylistDTO`.

A classe `DadosImportacao` tem como objetivo representar a estrutura de dados utilizada no processo de importação em lote de informações para o sistema. Serve como um contêiner que agrega listas de objetos transferidos (DTOs), permitindo a leitura e manipulação conjunta de utilizadores, músicas e playlists a partir de fontes externas, como ficheiros JSON ou bases de dados.

Cada campo da classe é público, facilitando o acesso direto durante operações de serialização e desserialização. Esta classe pode ser facilmente estendida para incluir outros tipos de dados, como álbuns ou subscrições, tornando-a uma estrutura flexível e central no processo de carregamento de dados iniciais para o sistema.

2.42 InputUtil

Variáveis de Instância:

- **(nenhuma)** A classe não possui variáveis de instância. Todos os métodos são estáticos e destinam-se a facilitar a leitura de diferentes tipos de dados a partir da entrada do utilizador via terminal.

A classe `InputUtil` fornece um conjunto de métodos utilitários para leitura segura e interativa de dados a partir do terminal, utilizando a classe `Scanner`. O seu principal objetivo é validar e normalizar entradas do utilizador, garantindo a robustez e usabilidade da interface textual.

- `public static int lerOpcao(Scanner scanner)` Solicita ao utilizador a introdução de uma opção numérica. Repetirá a leitura até que um valor inteiro válido seja introduzido.

- `public static String lerTexto(Scanner scanner, String prompt)` Exibe uma mensagem personalizada e lê uma linha de texto, removendo espaços em branco no início e fim.

- `public static int lerInteiro(Scanner scanner, String prompt)` Solicita um número inteiro ao utilizador. Repetirá a leitura até que o valor introduzido seja válido, tratando exceções de conversão.

- `public static double lerDecimal(Scanner scanner, String prompt)` Lê um número decimal (ponto flutuante). O método valida a entrada e garante que o utilizador introduza um valor do tipo `double`.

- `public static boolean lerBooleano(Scanner scanner, String prompt)` Lê uma resposta binária do tipo sim/não, aceitando "S" ou "N". Utiliza letras maiúsculas para normalização da entrada.

- `public static boolean lerBoolean(Scanner scanner, String mensagem)` Variante do método anterior, aceita respostas como "sim", "não", "s", "n", "nao", "não" (em letras minúsculas). Permite uma interpretação mais flexível da resposta do utilizador.

A classe `InputUtil` é essencial para padronizar a entrada de dados em aplicações de consola, evitando erros comuns de parsing e garantindo uma experiência de utilização mais fluida e intuitiva. É especialmente útil em menus interativos e fluxos de entrada que exigem validação e repetição até a resposta ser válida.

2.43 MusicaDTO

Variáveis de Instância:

- **public String nome;** Armazena o nome da música. Corresponde ao título da faixa musical.
- **public String interprete;** Guarda o nome do intérprete da música, ou seja, o artista ou banda responsável pela execução da faixa.
- **public String editora;** Indica a editora responsável pela produção ou distribuição da música.
- **public String letra;** Contém a letra da música, normalmente apresentada durante a reprodução ou para consulta do utilizador.
- **public String genero;** Especifica o género musical da música (por exemplo, pop, rock, jazz, etc.).
- **public int duracao;** Representa a duração da música em segundos. Esta informação pode ser utilizada para controlo e estatísticas de reprodução.
- **public boolean explicita;** Indica se a música contém conteúdo explícito (por exemplo, linguagem inapropriada ou temas sensíveis).

A classe `MusicaDTO` (Data Transfer Object) é utilizada para transportar os dados de uma música entre diferentes camadas do sistema, como processos de importação/exportação, serialização de ficheiros ou comunicação com interfaces externas. Esta estrutura simples e acessível facilita a transferência de informação, servindo de modelo temporário antes da conversão para objetos de domínio mais complexos, como a classe `Musica`.

2.44 Persistencia

Variáveis de Instância:

- **(nenhuma)** A classe não possui variáveis de instância. Todos os métodos são estáticos e estão relacionados com operações genéricas de leitura e escrita de objetos em ficheiros.

A classe `Persistencia` fornece métodos utilitários para persistência de dados, permitindo guardar e carregar listas de objetos em ficheiros através de serialização. Esta funcionalidade é fundamental para manter dados entre execuções da aplicação, como utilizadores, músicas, playlists, entre outros.

- `public static <T> void guardar(String ficheiro, List<T> lista)` Serializa uma lista de objetos do tipo genérico `T` e guarda o conteúdo num ficheiro especificado. Usa `ObjectOutputStream` para gravar os dados. Pode lançar `IOException` em caso de erro de escrita ou acesso ao ficheiro.

- `public static <T> List<T> carregar(String ficheiro)` Lê e desserializa uma lista de objetos do tipo `T` a partir de um ficheiro. Usa `ObjectInputStream` e requer que os objetos armazenados implementem a interface `Serializable`. Pode lançar `IOException` e `ClassNotFoundException` em caso de erro de leitura ou incompatibilidade de tipos.

A classe `Persistencia` é genérica e reutilizável, podendo ser utilizada para guardar qualquer tipo de dados que sejam serializáveis. Esta abordagem simplifica o processo de armazenamento e recuperação de dados estruturados, promovendo a separação de responsabilidades e a reutilização de código em aplicações Java.

2.45 PlaylistDTO

Variáveis de Instância:

- **public String nome;** Armazena o nome da playlist. Serve como identificador e título apresentado ao utilizador.

- **public String criador;** Guarda o nome do utilizador que criou a playlist. Pode ser utilizado para atribuição de propriedade ou exibição de autoria.

- **public List<String> musicas;** Contém a lista de identificadores ou nomes das músicas incluídas na playlist. A ordem dos elementos nesta lista representa a ordem de reprodução.

- **public boolean publica;** Indica se a playlist é pública (visível para outros utilizadores) ou privada (acesso restrito ao criador).

A classe `PlaylistDTO` (Data Transfer Object) tem como objetivo representar uma estrutura simplificada de uma playlist para fins de transferência de dados. É especialmente útil em operações de importação/exportação, serialização ou comunicação entre camadas da aplicação. Este tipo de classe atua como intermediário entre formatos externos (como ficheiros JSON ou XML) e as entidades de domínio internas, facilitando a manipulação, validação e persistência de dados de forma desacoplada.

2.46 UtilizadorDTO

Variáveis de Instância:

- **public String nome;** Armazena o nome do utilizador. Identifica a pessoa no sistema.

- **public String email;** Guarda o endereço de e-mail do utilizador, utilizado para autenticação e comunicação.

- **public String morada;** Contém a morada (endereço) do utilizador, podendo ser usada para fins administrativos ou de personalização.
- **public String tipo;** Indica o tipo de utilizador (por exemplo, "normal", "premium"). Esta informação pode determinar o acesso a funcionalidades específicas do sistema.
- **public String senha;** Armazena a senha de acesso do utilizador, necessária para autenticação no sistema.
- **public List<ReproducaoDTO> historicoReproducoes;** Lista de objetos do tipo `ReproducaoDTO`, representando o histórico de músicas reproduzidas pelo utilizador.

A classe `UtilizadorDTO` funciona como um Data Transfer Object que encapsula os dados essenciais de um utilizador, geralmente usados em processos de importação/exportação ou comunicação entre componentes do sistema. A estrutura é simples e direta, facilitando a serialização e desserialização, bem como a transformação em objetos de domínio mais completos.

2.47 ReproducaoDTO

Variáveis de Instância:

- **public String musica;** Nome ou identificador da música reproduzida.
- **public String dataHora;** Data e hora da reprodução da música, armazenada como `String` para facilitar o transporte e leitura simples.
- **public int duracaoReproduzida;** Duração (em segundos) da reprodução efetuada da música. Pode representar uma reprodução total ou parcial.

A classe `ReproducaoDTO` representa um registo individual no histórico de reproduções de um utilizador. É utilizada dentro de `UtilizadorDTO` para organizar e armazenar os dados de uso do sistema, como parte da informação pessoal de cada utilizador. Esta estrutura permite a análise de padrões de reprodução e estatísticas relacionadas ao consumo de conteúdo musical.

3 Diagrama de Classes

Devido à quantidade de classes desenvolvidas ao longo do projeto, o diagrama de classes gerado tornou-se demasiado extenso e visualmente denso, dificultando a sua leitura completa numa única visualização. Por esse motivo, optámos por anexar as tabelas correspondentes a cada classe, de forma a facilitar a compreensão da estrutura e das relações entre os diferentes componentes do sistema.

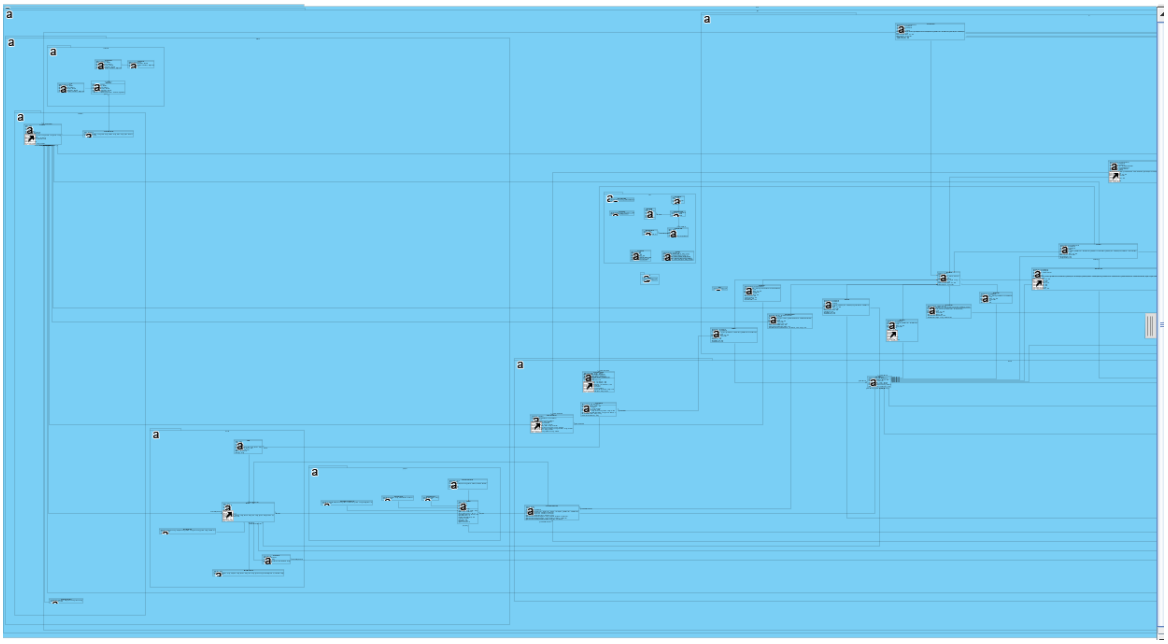


Figura 1: Diagrama de Classes — 1ª Parte

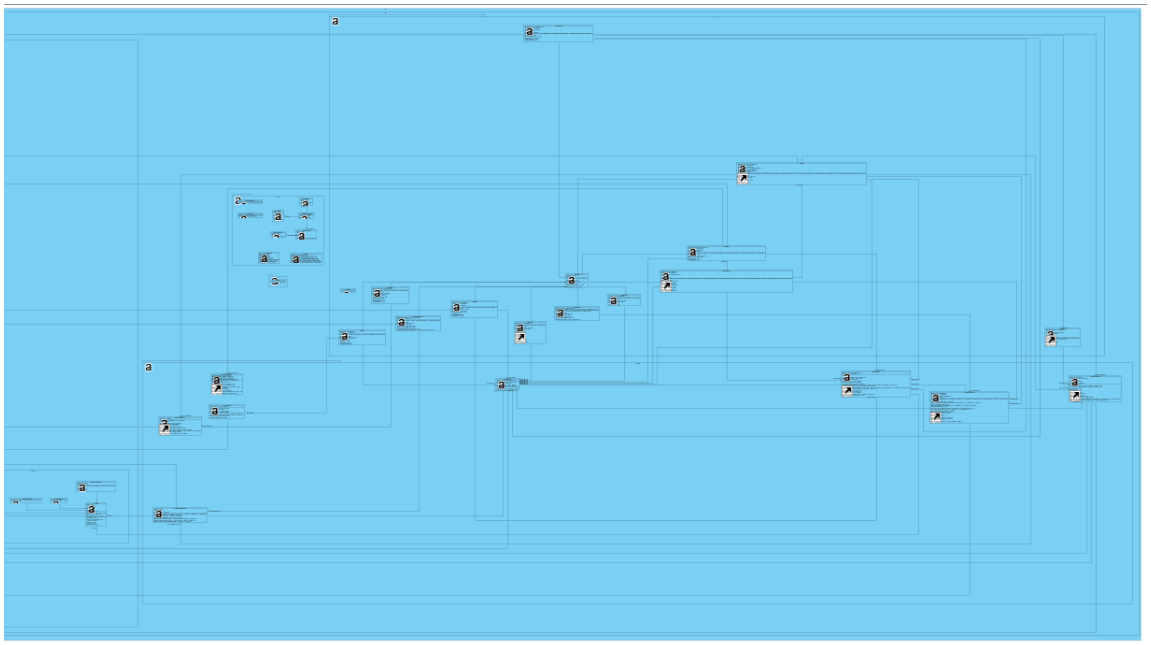


Figura 2: Diagrama de Classes — 2ª Parte

4 Conclusão

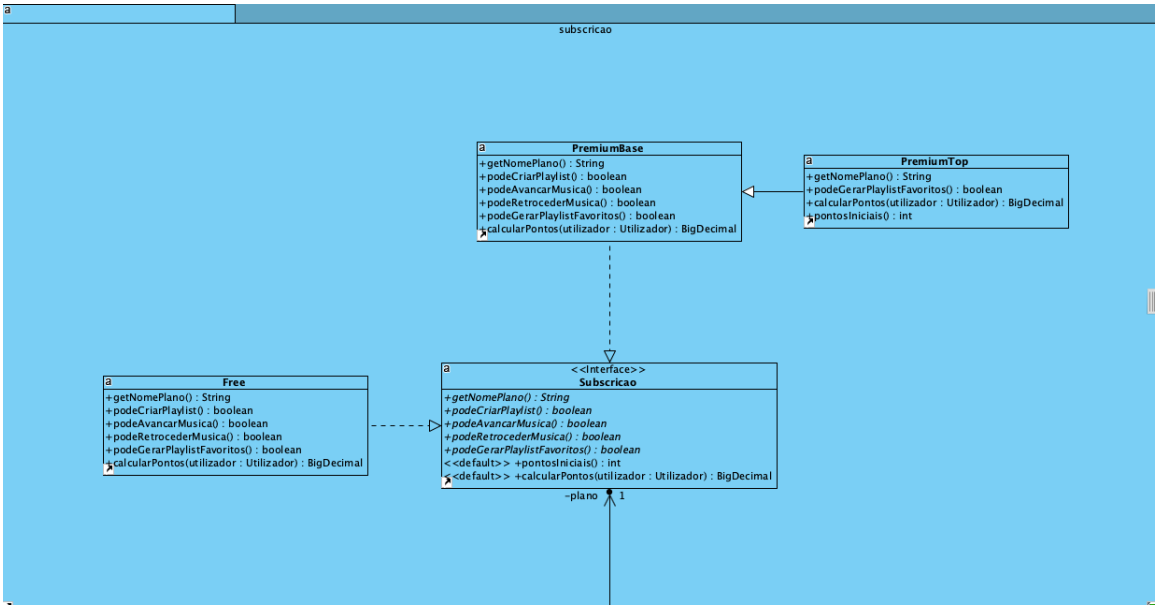
A realização deste projeto constituiu um desafio importante para o grupo, tanto no plano técnico como na organização do trabalho em equipa. Ao longo do desenvolvimento, enfrentámos diversos obstáculos, desde dificuldades na estruturação do código até à integração de diferentes componentes do sistema.

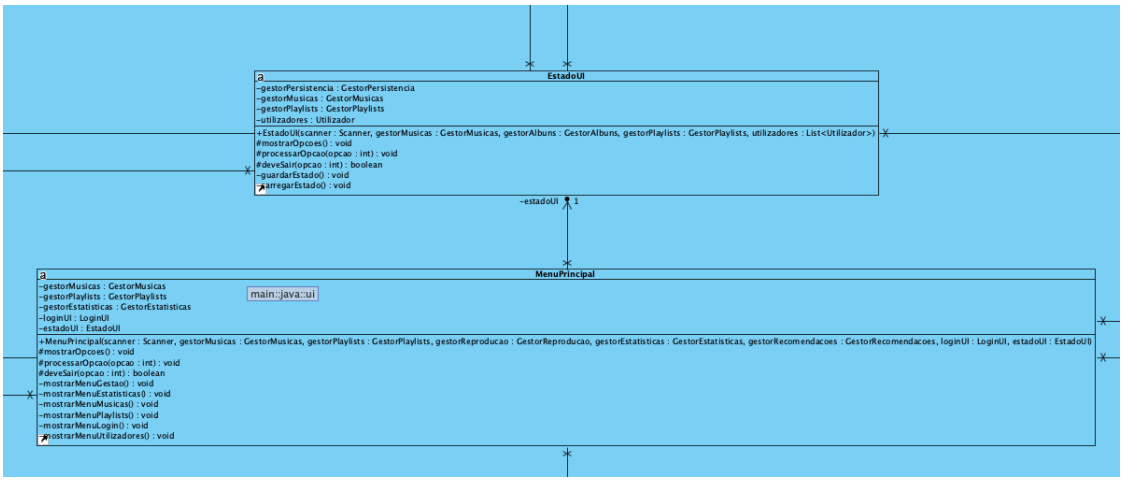
Embora nem sempre tenha sido fácil encontrar soluções imediatas, o processo de tentativa e erro revelou-se essencial para a nossa aprendizagem. Cada problema identificado permitiu-nos aprofundar o nosso conhecimento e melhorar o projeto de forma progressiva.

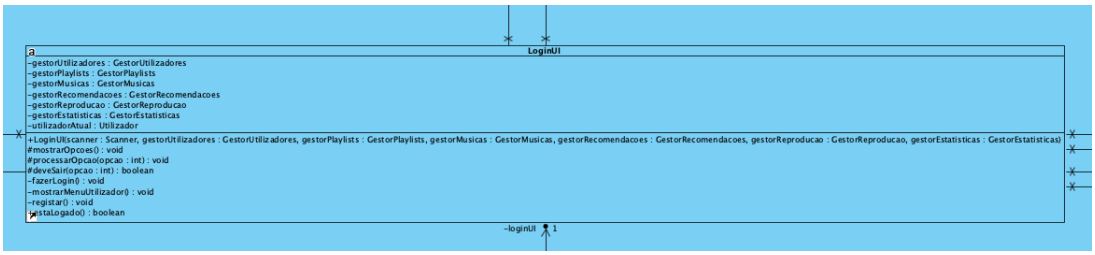
O trabalho em grupo exigiu coordenação, comunicação constante e divisão eficaz de tarefas, competências fundamentais não só no contexto académico, mas também no ambiente profissional. Através da colaboração, conseguimos consolidar ideias, ultrapassar dificuldades e chegar a uma solução funcional e coerente.

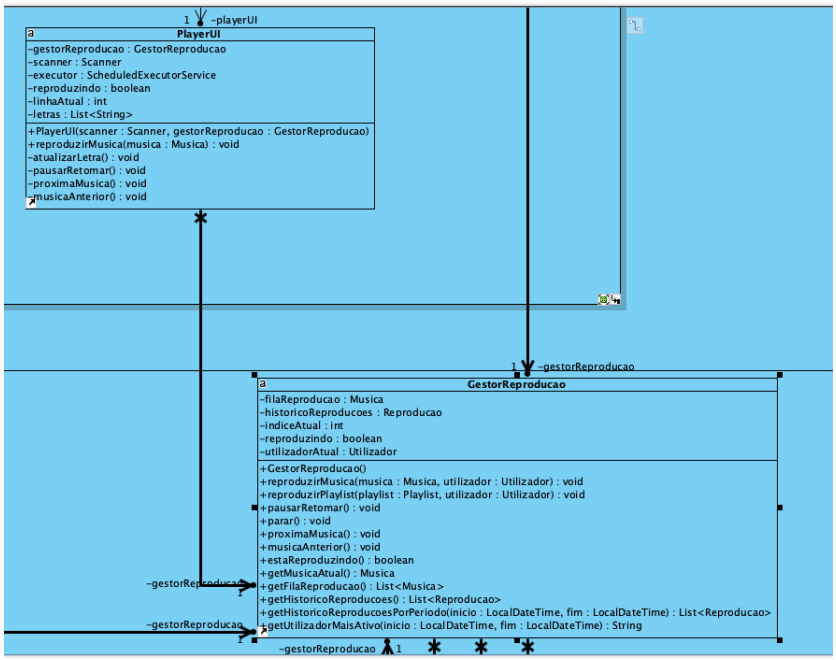
Em suma, este projeto foi uma oportunidade valiosa para aplicar conteúdos aprendidos ao longo da unidade curricular, mas também para desenvolver capacidades práticas e interpessoais. Acima de tudo, permitiu-nos crescer enquanto estudantes e futuros profissionais, mostrando que, mesmo perante desafios, é possível aprender, adaptar e evoluir.

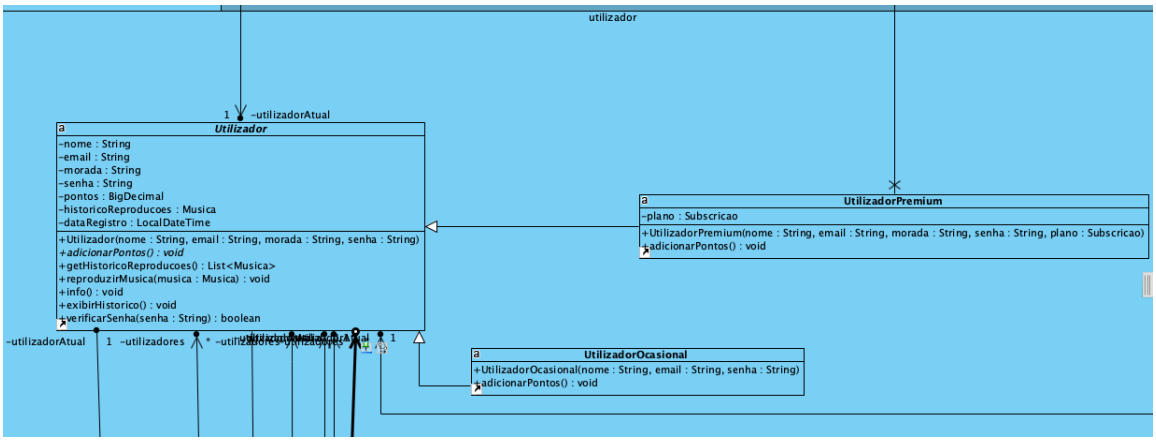
Anexos

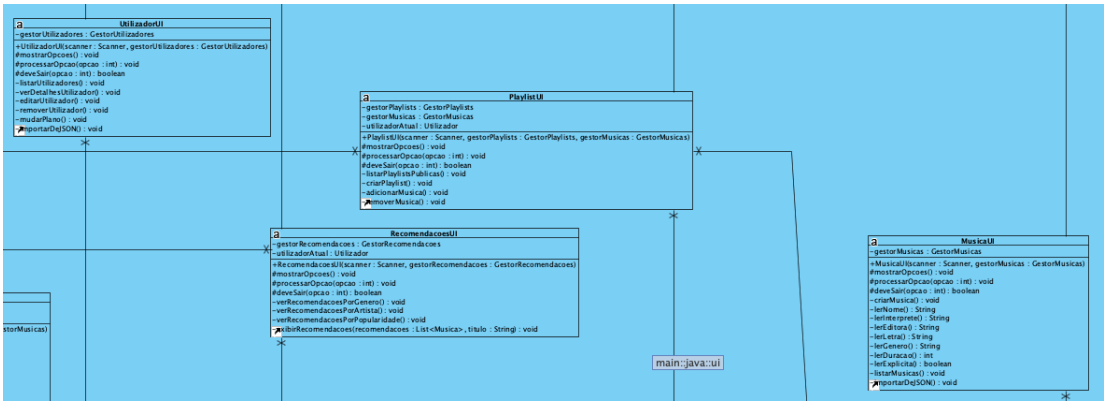

subscricao
PremiumBase
+getNomePlano() : String
+podeCriarPlaylist() : boolean
+podeAvancarMusica() : boolean
+podeRetrocederMusica() : boolean
+podeGerarPlaylistFavoritos() : boolean
+calcularPontos(utilizador : Utilizador) : BigDecimal
PremiumTop
+getNomePlano() : String
+podeGerarPlaylistFavoritos() : boolean
+calcularPontos(utilizador : Utilizador) : BigDecimal
+pontosIniciais() : int
Free
+getNomePlano() : String
+podeCriarPlaylist() : boolean
+podeAvancarMusica() : boolean
+podeRetrocederMusica() : boolean
+podeGerarPlaylistFavoritos() : boolean
+calcularPontos(utilizador : Utilizador) : BigDecimal
<<Interface>>
Subscricao
+getNomePlano() : String
+podeCriarPlaylist() : boolean
+podeAvancarMusica() : boolean
+podeRetrocederMusica() : boolean
+podeGerarPlaylistFavoritos() : boolean
<<default>> +pontosIniciais() : int
<<default>> +calcularPontos(utilizador : Utilizador) : BigDecimal
-plano
1

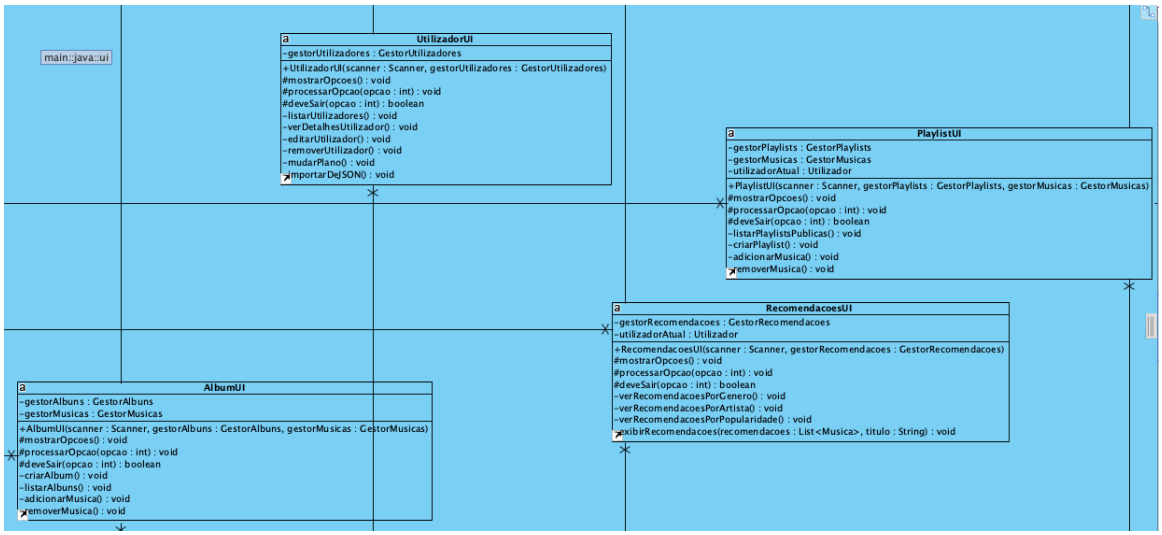

EstadoUI
-gestorPersistencia : GestorPersistencia
-gestorMusicas : GestorMusicas
-gestorPlaylists : GestorPlaylists
-utilizadores : Utilizador
+EstadoUI(scanner : Scanner, gestorMusicas : GestorMusicas, gestorAlbuns : GestorAlbuns, gestorPlaylists : GestorPlaylists, utilizadores : List<Utilizador>)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-guardarEstado() : void
-carregarEstado() : void
-estadoUI
1
MenuPrincipal
-gestorMusicas : GestorMusicas
-gestorPlaylists : GestorPlaylists
-gestorEstatisticas : GestorEstatisticas
-loginUI : LoginUI
-estadoUI : EstadoUI
main::java::ui
+MenuPrincipal(scanner : Scanner, gestorMusicas : GestorMusicas, gestorPlaylists : GestorPlaylists, gestorReproducao : GestorReproducao, gestorEstatisticas : GestorEstatisticas, gestorRecomendacoes : GestorRecomendacoes, loginUI : LoginUI, estadoUI : EstadoUI)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-mostrarMenuGestao() : void
-mostrarMenuEstatisticas() : void
-mostrarMenuMusicas() : void
-mostrarMenuPlaylists() : void
-mostrarMenuLogin() : void
-mostrarMenuUtilizadores() : void

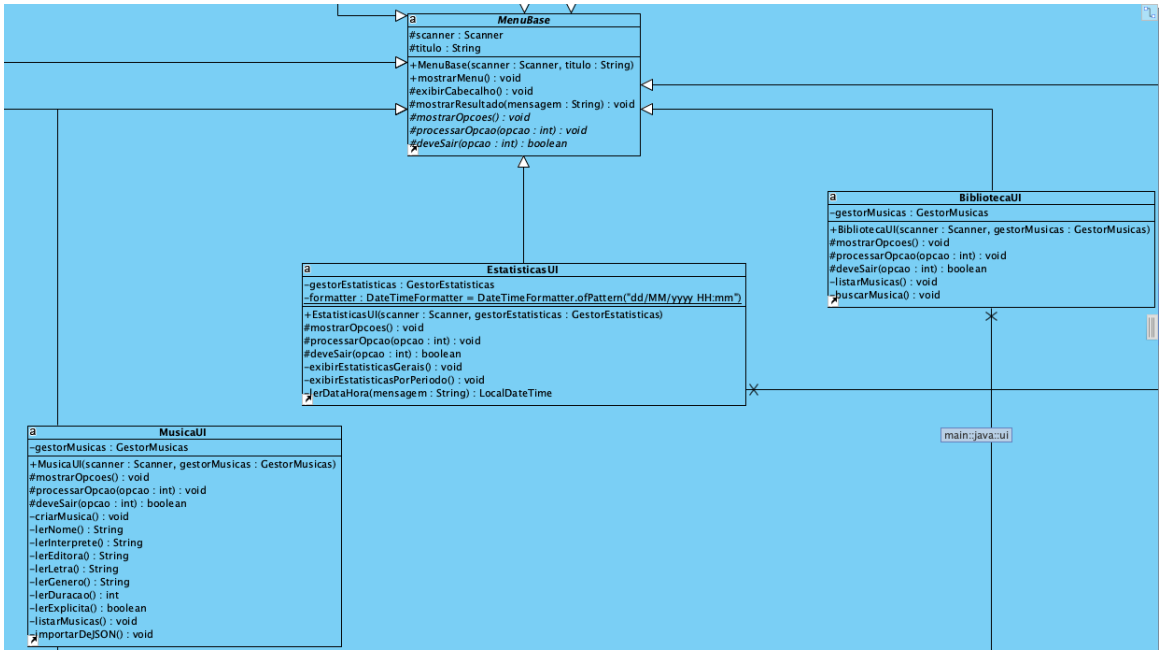

LoginUI
-gestorUtilizadores : GestorUtilizadores
-gestorPlaylists : GestorPlaylists
-gestorMusicas : GestorMusicas
-gestorRecomendacoes : GestorRecomendacoes
-gestorReproducao : GestorReproducao
-gestorEstatisticas : GestorEstatisticas
-utilizadorAtual : Utilizador
+LoginUI(scanner : Scanner, gestorUtilizadores : GestorUtilizadores, gestorPlaylists : GestorPlaylists, gestorMusicas : GestorMusicas, gestorRecomendacoes : GestorRecomendacoes, gestorReproducao : GestorReproducao, gestorEstatisticas : GestorEstatisticas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-fazerLogin() : void
-mostrarMenuUtilizador() : void
-registar() : void
-estaLogado() : boolean
-loginUI
1

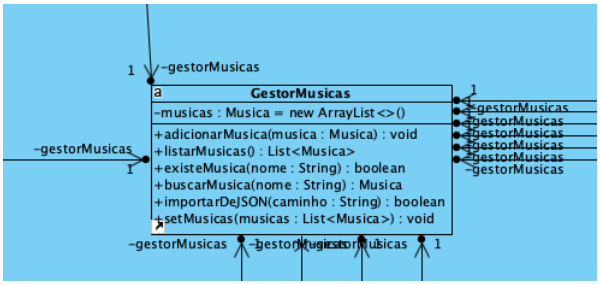

-playerUI
PlayerUI
-gestorReproducao : GestorReproducao
-scanner : Scanner
-executor : ScheduledExecutorService
-reproduzindo : boolean
-linhaAtual : int
-letras : List<String>
+PlayerUI(scanner : Scanner, gestorReproducao : GestorReproducao)
+reproduzirMusica(musica : Musica) : void
-atualizarLetra() : void
-pausarRetomar() : void
-proximaMusica() : void
-musicaAnterior() : void
-gestorReproducao
GestorReproducao
-filaReproducao : Musica
-historicoReproducoes : Reproducao
-indiceAtual : int
-reproduzindo : boolean
-utilizadorAtual : Utilizador
+GestorReproducao()
+reproduzirMusica(musica : Musica, utilizador : Utilizador) : void
+reproduzirPlaylist(playlist : Playlist, utilizador : Utilizador) : void
+pausarRetomar() : void
+parar() : void
+proximaMusica() : void
+musicaAnterior() : void
+estaReproduzindo() : boolean
+getMusicaAtual() : Musica
+getFilaReproducao() : List<Musica>
+getHistoricoReproducoes() : List<Reproducao>
+getHistoricoReproducoesPorPeriodo(inicio : LocalDateTime, fim : LocalDateTime) : List<Reproducao>
+getUtilizadorMaisAtivo(inicio : LocalDateTime, fim : LocalDateTime) : String
-gestorReproducao
-gestorReproducao
-gestorReproducao

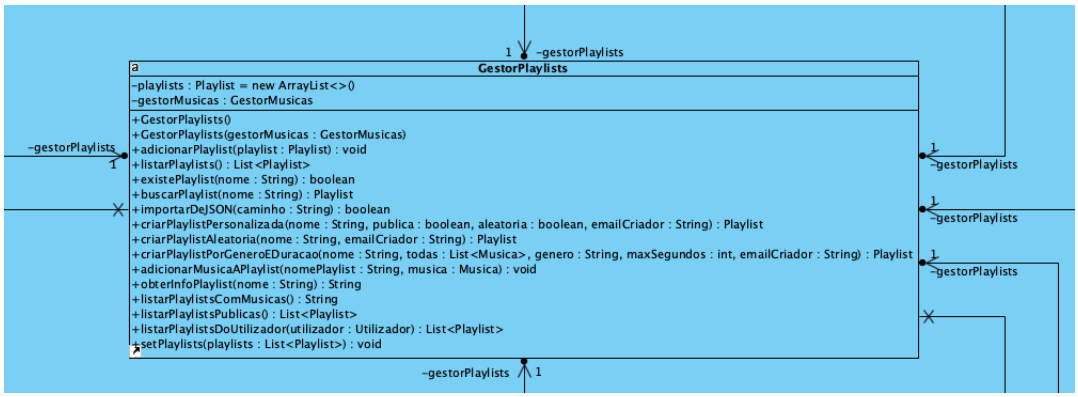

utilizador
-utilizadorAtual
Utilizador
-nome : String
-email : String
-morada : String
-senha : String
-pontos : BigDecimal
-historicoReproducoes : Musica
-dataRegistro : LocalDateTime
+Utilizador(nome : String, email : String, morada : String, senha : String)
+adicionarPontos() : void
+getHistoricoReproducoes() : List<Musica>
+reproduzirMusica(musica : Musica) : void
+info() : void
+exibirHistorico() : void
+verificarSenha(senha : String) : boolean
-utilizadorAtual
-utilizadores
UtilizadorPremium
-plano : Subscricao
+UtilizadorPremium(nome : String, email : String, morada : String, senha : String, plano : Subscricao)
+adicionarPontos() : void
UtilizadorOcasional
+UtilizadorOcasional(nome : String, email : String, senha : String)
+adicionarPontos() : void

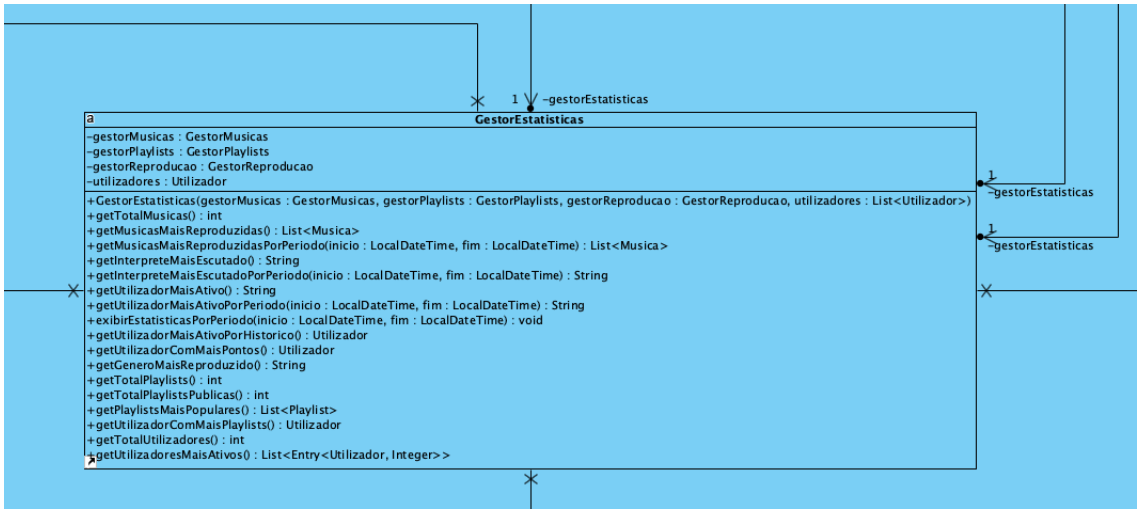

UtilizadorUI
-gestorUtilizadores : GestorUtilizadores
+UtilizadorUI(scanner : Scanner, gestorUtilizadores : GestorUtilizadores)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-listarUtilizadores() : void
-verDetalhesUtilizador() : void
-editarUtilizador() : void
-removerUtilizador() : void
-mudarPlano() : void
-exportarDejSON() : void
PlaylistUI
-gestorPlaylists : GestorPlaylists
-gestorMusicas : GestorMusicas
-utilizadorAtual : Utilizador
+PlaylistUI(scanner : Scanner, gestorPlaylists : GestorPlaylists, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-listarPlaylistsPublicas() : void
-criarPlaylist() : void
-adicionarMusica() : void
-removerMusica() : void
RecomendacoesUI
-gestorRecomendacoes : GestorRecomendacoes
-utilizadorAtual : Utilizador
+RecomendacoesUI(scanner : Scanner, gestorRecomendacoes : GestorRecomendacoes)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-verRecomendacoesPorGenero() : void
-verRecomendacoesPorArtista() : void
-verRecomendacoesPorPopularidade() : void
-exibirRecomendacoes(recomendacoes : List<Musica>, titulo : String) : void
MusicaUI
-gestorMusicas : GestorMusicas
+MusicaUI(scanner : Scanner, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-criarMusica() : void
-lerNome() : String
-lerInterprete() : String
-lerEditora() : String
-lerLetra() : String
-lerGenero() : String
-lerDuracao() : int
-lerExplicita() : boolean
-listarMusicas() : void
-exportarDejSON() : void
main::java::ui

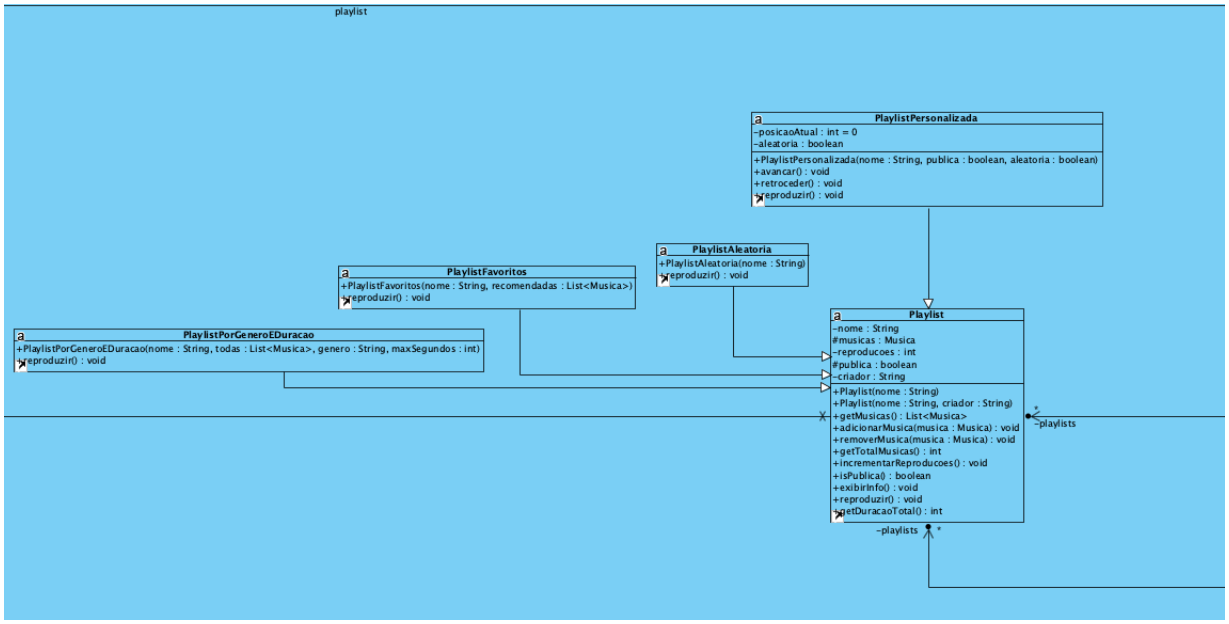

main::java::ui
UtilizadorUI
-gestorUtilizadores : GestorUtilizadores
+UtilizadorUI(scanner : Scanner, gestorUtilizadores : GestorUtilizadores)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-listarUtilizadores() : void
-verDetalhesUtilizador() : void
-editarUtilizador() : void
-removerUtilizador() : void
-mudarPlano() : void
-importarDeJSON() : void
PlaylistUI
-gestorPlaylists : GestorPlaylists
-gestorMusicas : GestorMusicas
-utilizadorAtual : Utilizador
+PlaylistUI(scanner : Scanner, gestorPlaylists : GestorPlaylists, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-listarPlaylistsPublicas() : void
-criarPlaylist() : void
-adicionarMusica() : void
-removerMusica() : void
RecomendacoesUI
-gestorRecomendacoes : GestorRecomendacoes
-utilizadorAtual : Utilizador
+RecomendacoesUI(scanner : Scanner, gestorRecomendacoes : GestorRecomendacoes)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-verRecomendacoesPorGenero() : void
-verRecomendacoesPorArtista() : void
-verRecomendacoesPorPopularidade() : void
-exibirRecomendacoes(recomendacoes : List<Musica>, titulo : String) : void
AlbumUI
-gestorAlbuns : GestorAlbuns
-gestorMusicas : GestorMusicas
+AlbumUI(scanner : Scanner, gestorAlbuns : GestorAlbuns, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-criarAlbum() : void
-listarAlbuns() : void
-adicionarMusica() : void
-removerMusica() : void


MenuBase
#scanner : Scanner
#titulo : String
+MenuBase(scanner : Scanner, titulo : String)
+mostrarMenu() : void
#exibirCabecalho() : void
#mostrarResultado(mensagem : String) : void
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
BibliotecaUI
-gestorMusicas : GestorMusicas
+BibliotecaUI(scanner : Scanner, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-listarMusicas() : void
-buscarMusica() : void
EstatisticasUI
-gestorEstatisticas : GestorEstatisticas
-formatter : DateTimeFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm")
+EstatisticasUI(scanner : Scanner, gestorEstatisticas : GestorEstatisticas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-exibirEstatisticasGerais() : void
-exibirEstatisticasPorPeriodo() : void
-lerDataHora(mensagem : String) : LocalDateTime
MusicaUI
-gestorMusicas : GestorMusicas
+MusicaUI(scanner : Scanner, gestorMusicas : GestorMusicas)
#mostrarOpcoes() : void
#processarOpcao(opcao : int) : void
#deveSair(opcao : int) : boolean
-criarMusica() : void
-lerNome() : String
-lerInterprete() : String
-lerEditora() : String
-lerLetra() : String
-lerGenero() : String
-lerDuracao() : int
-lerExplicita() : boolean
-listarMusicas() : void
-importarDeJSON() : void
main::java::ui


-gestorMusicas
GestorMusicas
-musicas : Musica = new ArrayList<>()
+adicionarMusica(musica : Musica) : void
+listarMusicas() : List<Musica>
+existeMusica(nome : String) : boolean
+buscarMusica(nome : String) : Musica
+importarDeJSON(caminho : String) : boolean
+setMusicas(musicas : List<Musica>) : void
-gestorMusicas
1


GestorPlaylists
-playlists : Playlist = new ArrayList<>()
-gestorMusicas : GestorMusicas
+GestorPlaylists()
+GestorPlaylists(gestorMusicas : GestorMusicas)
+adicionarPlaylist(playlist : Playlist) : void
+listarPlaylists() : List<Playlist>
+existePlaylist(nome : String) : boolean
+buscarPlaylist(nome : String) : Playlist
+importarDeJSON(caminho : String) : boolean
+criarPlaylistPersonalizada(nome : String, publica : boolean, aleatoria : boolean, emailCriador : String) : Playlist
+criarPlaylistAleatoria(nome : String, emailCriador : String) : Playlist
+criarPlaylistPorGeneroEDuracao(nome : String, todas : List<Musica>, genero : String, maxSegundos : int, emailCriador : String) : Playlist
+adicionarMusicaAPlaylist(nomePlaylist : String, musica : Musica) : void
+obterInfoPlaylist(nome : String) : String
+listarPlaylistsComMusicas() : String
+listarPlaylistsPublicas() : List<Playlist>
+listarPlaylistsDoUtilizador(utilizador : Utilizador) : List<Playlist>
+setPlaylists(playlists : List<Playlist>) : void
-gestorPlaylists
1
-gestorPlaylists
1
-gestorPlaylists
1
-gestorPlaylists
1
-gestorPlaylists
1
-gestorPlaylists
1


GestorEstatisticas
-gestorMusicas : GestorMusicas
-gestorPlaylists : GestorPlaylists
-gestorReproducao : GestorReproducao
-utilizadores : Utilizador
+GestorEstatisticas(gestorMusicas : GestorMusicas, gestorPlaylists : GestorPlaylists, gestorReproducao : GestorReproducao, utilizadores : List<Utilizador>)
+getTotalMusicas() : int
+getMusicasMaisReproduzidas() : List<Musica>
+getMusicasMaisReproduzidasPorPeriodo(inicio : LocalDateTime, fim : LocalDateTime) : List<Musica>
+getInterpreteMaisEscutado() : String
+getInterpreteMaisEscutadoPorPeriodo(inicio : LocalDateTime, fim : LocalDateTime) : String
+getUtilizadorMaisAtivo() : String
+getUtilizadorMaisAtivoPorPeriodo(inicio : LocalDateTime, fim : LocalDateTime) : String
+exibirEstatisticasPorPeriodo(inicio : LocalDateTime, fim : LocalDateTime) : void
+getUtilizadorMaisAtivoPorHistorico() : Utilizador
+getUtilizadorComMaisPontos() : Utilizador
+getGeneroMaisReproduzido() : String
+getTotalPlaylists() : int
+getTotalPlaylistsPublicas() : int
+getPlaylistsMaisPopulares() : List<Playlist>
+getUtilizadorComMaisPlaylists() : Utilizador
+getTotalUtilizadores() : int
+getUtilizadoresMaisAtivos() : List<Entry<Utilizador, Integer>>
1
-gestorEstatisticas
1
-gestorEstatisticas
1
-gestorEstatisticas


playlist
PlaylistPersonalizada
-posicaoAtual : int = 0
-aleatoria : boolean
+PlaylistPersonalizada(nome : String, publica : boolean, aleatoria : boolean)
+avancar() : void
+retroceder() : void
+reproduzir() : void
PlaylistAleatoria
+PlaylistAleatoria(nome : String)
+reproduzir() : void
PlaylistFavoritos
+PlaylistFavoritos(nome : String, recomendadas : List<Musica>)
+reproduzir() : void
PlaylistPorGeneroEDuracao
+PlaylistPorGeneroEDuracao(nome : String, todas : List<Musica>, genero : String, maxSegundos : int)
+reproduzir() : void
Playlist
-nome : String
#musicas : Musica
-reproducoes : int
#publica : boolean
-criador : String
+Playlist(nome : String)
+Playlist(nome : String, criador : String)
+getMusicas() : List<Musica>
+adicionarMusica(musica : Musica) : void
+removerMusica(musica : Musica) : void
+getTotalMusicas() : int
+incrementarReproducoes() : void
+isPublica() : boolean
+exibirInfo() : void
+reproduzir() : void
+getDuracaoTotal() : int
-playlists
*
-playlists
*